A project report on

QUANTUM-INSPIRED EDGE PRUNING AND TRUST-AWARE GRAPH NEURAL INFERENCE FOR TASK ALLOCATION IN DYNAMIC ROVER NETWORKS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

NANDANA SATISH (23BAI1568)
P PREETHAM (23BRS1346)
BIRESH BISWAS (23BRS1255)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

A project report on

QUANTUM-INSPIRED EDGE PRUNING AND TRUST-AWARE GRAPH NEURAL INFERENCE FOR TASK ALLOCATION IN DYNAMIC ROVER NETWORKS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

NANDANA SATISH (23BAI1568)
P PREETHAM (23BRS1346)
BIRESH BISWAS (23BRS1255)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Quantum-Inspired Edge Pruning and Trust-Aware Graph Neural Inference for Task Allocation in Dynamic Rover Networks" submitted by Nandana Satish (23BAI1568), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Selvam D.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date:

Signature of the Candidate

Nandana Satish

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Quantum-Inspired Edge Pruning and Trust-Aware Graph Neural Inference for Task Allocation in Dynamic Rover Networks" is prepared and submitted by Nandana Satish (23BAI1568) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Selvam D

Date:

Signature of the Examiner	**Signature of the Examiner**
Name:	Name:
Date:	Date:

Approved by the Head of Department,
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Name: Dr. Tulasi Prasad Sariki
Date:

(Seal of SCHOOL YOU BELONG TO)

ABSTRACT

Rover teams coordinate over low-power radio links whose quality changes from second to second, so the graph of who can talk to whom is a moving target. This project builds and audits a pipeline for allocating tasks over that graph. Each cycle, a constrained quadratic unconstrained binary optimisation (QUBO) model prunes the graph to a working subgraph that respects per-rover link caps, a processing budget and exact connectivity of the mission rovers; a lightweight trust-aware graph neural network scores the rovers on the pruned graph; a greedy stage assigns the tasks; and an online trust model learns from the outcomes. The encoding agrees with an independent re-derivation to within 2.3×10^{-10}. Four annealing-family samplers find the exact optimum on only 48 to 64% of verifiable instances, whereas a feasibility-projected annealer developed here is exact on all of them and matches exact mixed-integer optima on 95% of larger ones; an exact mixed-integer solver is, however, about 100 times faster, so the QUBO is positioned as a quantum-hardware encoding. In a causal closed-loop simulator, trust-aware assignment beats a trust-only baseline by 8.1 percentage points (95% CI $[+4.2, +11.7]$, $p = 0.016$) and random assignment by 13.8, while the QUBO pruner, the only arm with zero constraint violations, trades 2 to 4 points of raw success for its guarantees. The pipeline also runs in a containerised Gazebo and ROS 2 world against live odometry, and in a purpose-built interactive 3D engine with real-time differential-drive rovers and live paired comparisons.

Keywords: QUBO, simulated annealing, graph neural networks, trust, rover swarms, fog computing, graph pruning, task allocation, Gazebo, ROS 2, real-time 3D simulation.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Selvam D, Assistant Professor Senior Grade I, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Quantum Computing and Theoretical Computer Science.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. Tulasi Prasad Sariki, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai
Date: **Nandana Satish**

CONTENTS

ABSTRACT . . . . . **i**

ACKNOWLEDGEMENT . . . . . **ii**

CONTENTS . . . . . **iii**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vii**

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE . . . . **viii**

CHAPTER 1 INTRODUCTION **1**
1.1 INTRODUCTION 1
1.2 OVERVIEW OF DYNAMIC ROVER NETWORKS 1
1.3 CHALLENGES 2
1.4 PROJECT STATEMENT 2
1.5 OBJECTIVES 3
1.6 SCOPE 3
1.7 CONTRIBUTIONS 3
1.8 ORGANIZATION OF THE REPORT 4

CHAPTER 2 BACKGROUND AND LITERATURE SURVEY **5**
2.1 INTRODUCTION 5
2.2 GRAPH NEURAL NETWORKS 5
2.3 QUBO MODELS AND THE PENALTY METHOD 5
2.4 ANNEALING, TABU SEARCH AND EXACT SOLVERS 6
2.5 FOG COMPUTING, TRUST AND TASK ALLOCATION 6
2.6 ROBOT SIMULATION AND REAL-TIME RENDERING 7
2.7 SURVEY OF RELATED WORK 7
2.8 RESEARCH GAP 8

CHAPTER 3 SYSTEM MODEL AND ARCHITECTURE **9**
3.1 DESIGN GOALS 9
3.2 THREE-TIER DEPLOYMENT 9
3.3 GRAPH MODEL AND FEATURES 9
3.4 ONE PIPELINE CYCLE 11
3.5 THE MISSION MODEL 11
3.6 SOFTWARE ORGANISATION 12
3.7 PHYSICAL TESTBED CONCEPT 13

CHAPTER 4 CONSTRAINED QUBO FORMULATION **15**
4.1 OVERVIEW 15
4.2 DECISION VARIABLES AND NOTATION 15
4.3 THE ENERGY FUNCTION 16
4.4 THE SOFT TERMS 16

4.5 ENCODING INEQUALITIES WITH SLACK 17
4.6 THE BUDGET CONSTRAINT 17
4.7 EXACT CONNECTIVITY BY SINGLE-COMMODITY FLOW . . . 18
4.8 WHY THE PENALTIES DOMINATE 18
4.9 A WORKED EXAMPLE ON FIVE ROVERS 18
4.10 VERIFICATION OF THE ENCODING 20
4.11 PROBLEM SIZE 20

CHAPTER 5 SOLVING THE QUBO: THE PENALTY BARRIER AND PROJECTED ANNEALING 22
5.1 ANNEALING A PENALTY-ENCODED QUBO 22
5.2 THE PENALTY BARRIER 22
5.3 BENCHMARKING FOUR SERIOUS SAMPLERS 23
5.4 PROJECTED EDGE ANNEALING 24
5.5 HOW GOOD IS THE PROJECTED ANNEALER? 25
5.6 SENSITIVITY TO THE ENCODING'S FREE PARAMETERS . . . 25
5.7 THE RELAXATION LADDER 25
5.8 IMPLICATIONS 26

CHAPTER 6 EXACT CLASSICAL BASELINES 27
6.1 WHY AN EXACT BASELINE 27
6.2 THE DIRECT MIXED-INTEGER PROGRAM 27
6.3 THE EXACT LINEARISED SOLVE OF THE Q MATRIX 28
6.4 CROSS-VALIDATION OF THE REFEREES 28
6.5 RESULTS 29
6.6 POSITIONING THE QUBO 30

CHAPTER 7 TRUST-AWARE GNN INFERENCE AND TASK ASSIGNMENT 31
7.1 ROLE OF THE INFERENCE STAGE 31
7.2 NODE FEATURES AND MESSAGE WEIGHTS 31
7.3 TWO-LAYER MESSAGE PASSING 31
7.4 GATES, CRITICALITY AND DELIVERY AWARENESS 32
7.5 CAPACITY-AWARE GREEDY ASSIGNMENT 32
7.6 THE ONLINE TRUST MODEL 33
7.7 EDGE LIFECYCLE 34
7.8 THE CLOUD-TRAINED READOUT 34
7.9 ABLATION ARMS 34
7.10 COMPUTATIONAL COST 34

CHAPTER 8 CLOSED-LOOP SIMULATION AND EVALUATION METHODOLOGY 36
8.1 PURPOSE AND DESIGN PRINCIPLES 36
8.2 THE WORLD MODEL 36
8.3 THE RADIO MODEL 37
8.4 FAULTS 37
8.5 EXPERIMENTAL ARMS 37
8.6 METRICS 38
8.7 STATISTICAL METHODOLOGY 39

8.8 SCALING AND PRESSURE EXPERIMENTS 39

CHAPTER 9 VERIFICATION AND TESTING 41
9.1 PHILOSOPHY: A HOSTILE AUDIT 41
9.2 THE SHARED TOOLBOX 41
9.3 THE FOURTEEN SECTIONS OF THE SUITE 41
9.4 SELECTED CHECKS IN DEPTH 43
9.5 TESTING THE DEPLOYMENTS 44
9.6 REPRODUCIBILITY PRACTICES 45

CHAPTER 10 RESULTS AND DISCUSSION 46
10.1 OVERVIEW 46
10.2 THE PRUNER COMPARISON 46
10.3 CONSTRAINT PRESSURE 48
10.4 ASSIGNMENT ABLATION 49
10.5 AMORTISATION AND SCALING 49
10.6 TRUST QUALITY, TRAINING AND FAULTS 50
10.7 DISCUSSION AND LIMITATIONS 50

CHAPTER 11 PHYSICS SIMULATION IN GAZEBO WITH ROS 2 . . . 52
11.1 MOTIVATION AND SOFTWARE STACK 52
11.2 THE WORLD AND THE ROVER 52
11.3 THE PLANNER NODE 53
11.4 PACKAGING AND LAUNCH 54
11.5 BEHAVIOUR AND LIMITATIONS 55

CHAPTER 12 AN INTERACTIVE 3D SIMULATION ENGINE 56
12.1 GOALS AND ARCHITECTURE 56
12.2 THE MOTION ENGINE 58
12.3 THE RENDERER 59
12.4 THE INTERFACE 61
12.5 EDGE CASES AND VERIFICATION 62

CHAPTER 13 CONCLUSION AND FUTURE WORK 64
13.1 CONCLUSION 64
13.2 KEY TAKE-AWAYS 65
13.3 PERFORMANCE AGAINST THE OBJECTIVES 65
13.4 BENEFITS 66
13.5 FUTURE WORK 66

APPENDICES 67
APPENDIX 1: PSEUDO-CODE OF THE PIPELINE, THE SOLVER AND THE MOTION ENGINE 67
APPENDIX 2: CONSTANTS AND SETTINGS 69

REFERENCES 71

LIST OF FIGURES

1.1 The pruning idea: of the links in range (a), a small connected subset is kept (b, solid) subject to link caps, a cost budget and connectivity of the mission rovers. 2

3.1 Three-tier deployment. 10
3.2 One pipeline cycle. 11
3.3 Physical testbed concept. 14

4.1 Energy ladder of the worked example. 20

5.1 Schematic (not to scale) of the penalty barrier. 23
5.2 Exact-solve rate of four annealing-family samplers on the actual Q matrices of 25 instances. 23

10.1 Mission success and constraint violations of the six pruners at a binding, feasible budget (0.75× the full-graph cost). 47
10.2 Offload rate, delivery-failure rate and delivery-critical recall of each pruner at the binding budget. 48
10.3 Budget pressure sweep: success by arm (left) and whether each arm honoured the budget (right). 49
10.4 Wall-clock time for each decision on the scaling instances. The time drops at $n = 30$ because connectivity is no longer enforced above 20 rovers. 50

11.1 The Gazebo integration inside one container. 52
11.2 Top view of the Gazebo world: with a 5 m radio range the four rovers form five candidate links; the end rovers are 6 m apart and cannot link. 53
11.3 Launch sequence of the container. 54

12.1 The engine running the twelve-rover *Dense swarm* preset. The mission controls are on the left and the 3D scene, with kept links and task beacons, is in the centre. The overview map and the radar for each rover are on the right, and the live comparison with trust-only assignment is along the bottom. 56
12.2 The eight-rover *Field survey* preset from the orbit camera (left) and the top camera (right): gold kept links, faint candidates, task beacons resolving to success (mint) or failure (orange), and the root's ring. . . 60
12.3 Left: chase camera behind the default rover (ceramic body, gold-foil deck, stereo camera, antenna light, treaded wheels with orange hubs). 60
12.4 Overview map, rover radars and the live comparison strip. 62

LIST OF TABLES

2.1 Qualitative comparison of related work with the present project. . . . 7

3.1 Edge and rover features used by the pruner and the GNN. 10
3.2 Software components and their responsibilities. 12

4.1 Variable families of the QUBO. Every integer is binary-encoded. . . . 15
4.2 Constants of the QUBO used throughout the report. 16
4.3 Coefficients generated by one degree constraint with $D_i = 2$ (before multiplication by ρ_d). 17
4.4 The five-rover worked example: utility U_e from Eq. (3.2), integer cost c_e and previous-cycle decision x_e^{t-1}. 19

5.1 The relaxation ladder of the production pruner, tried top to bottom. . . 26

6.1 Projected annealer and exact MILP compared on 20 instances. 29
6.2 Soft energy on the scaling instances. 29

8.1 Metrics computed by the simulator. 38

9.1 The fourteen sections of the test suite. 42

10.1 Scoreboard of the main quantitative results. 46
10.2 Pruner comparison, 10 rovers, 20 cycles, 16 seeds, fixed budget $K = 15$. 46
10.3 Constraint-pressure sweep, 10 rovers, 20 cycles, 16 seeds per cell; differences in percentage points with permutation p-values. 48
10.4 Ablation of the assignment stage with 10 rovers, 20 cycles and 8 seeds. Differences are computed per seed, in pairs. 49
10.5 Solving in the cloud every k cycles, with 10 rovers, 20 cycles and 6 seeds. 50

11.1 Settings of the Gazebo world and of the rover model. 53
11.2 Planner cycles with the rovers at their spawn positions. 55

12.1 Tunable parameters, their ranges and scopes. 57
12.2 The presets and the planner time per cycle we measured for each. . . . 58
12.3 Edge cases handled by the engine. 63

13.1 Outcome of each objective. 65
A2.1 Constants and default settings used in this report. 69

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviations

AUC	Area Under the receiver-operating-characteristic Curve
BKS	Best-Known Solution
CBC	COIN-OR Branch and Cut, an open-source mixed-integer solver
CI	Confidence Interval
CPU	Central Processing Unit
DDS	Data Distribution Service, the transport beneath ROS 2
ESP-NOW	Low-power peer-to-peer wireless protocol for ESP32-class devices
EWMA	Exponentially Weighted Moving Average
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
MILP	Mixed-Integer Linear Program
MQTT	Message Queuing Telemetry Transport
MSAA	Multi-Sample Anti-Aliasing
ODE	Open Dynamics Engine, the physics engine used by the Gazebo world
PBR	Physically Based Rendering
QUBO	Quadratic Unconstrained Binary Optimisation
ROS 2	Robot Operating System, version 2
SA	Simulated Annealing
SDF	Simulation Description Format
SMAA	Subpixel Morphological Anti-Aliasing
SQA	Simulated Quantum Annealing
UI	User Interface
WebGL	Web Graphics Library, the browser's GPU interface

Symbols

$G_t = (V, E_t)$	Undirected communication graph of the fleet at cycle t
$n = \|V\|$	Number of rovers (fleet size)
T_i, B_i, ℓ_i, q_i	Trust, battery, load and capability of rover i
$T_e, L_e, B_e, P_e, A_e, C_e$	Edge trust, link quality, battery, proximity, freshness and task criticality
$\bar{c}_e,\ c_e$	Raw processing cost of edge e and its quantised integer value
U_e	Weighted utility of edge e
$H,\ O$	Hard-critical edges (always kept) and optional edges $O = E_t \setminus H$
$R,\ r,\ m$	Required (mission-critical) rovers, the root rover, and $m = \|R\| - 1$
$D_i,\ K$	Per-rover degree cap and global processing-cost budget
x_e	Binary decision: keep (1) or prune (0) edge e
$s_i,\ s_C,\ q_e$	Integer slack variables for degree, budget and flow capacity

f_{uv}	Integer flow on the directed arc (u, v)
$\lambda,\ \eta$	Sparsity penalty and topology-stability penalty
$\rho_d, \rho_C, \rho_F, \rho_{FC}$	Penalty weights for degree, budget, flow conservation and flow capacity
$z,\ Q$	Bit vector of all QUBO variables and the QUBO matrix
$H(z)$	QUBO energy $z^\top Qz$ + const
H^*	Exact (or best-known) optimal energy of an instance
$\Delta E,\ \tau_{\mathrm{SA}}$	Energy change of a move and annealing temperature
a_{ij}	Message weight $T_{ij}L_{ij}$ between rovers i and j
$h_i^{(k)}$	Hidden state of rover i after k message-passing rounds
$W_s,\ W_n,\ b$	Self weight, neighbour weight and bias of a message-passing layer
$w,\ \beta$	Weights and bias of the logistic readout
$\sigma(\cdot)$	Logistic sigmoid
$\alpha,\ \delta$	EWMA retention factor and trust-aging rate
p_i	Delivery factor: weakest link quality on rover i's widest path to the root
β_K	Budget factor: budget as a fraction of the full-graph cost
(x, z, ψ)	Planar pose of a rover in the 3D engine: position and heading
$v,\ \omega$	Forward speed and yaw rate of a differential-drive rover
$v_L,\ v_R$	Left and right wheel rim speeds
$L_w,\ r_w$	Wheel track (separation) and wheel radius
R_b	Radius of the circular collision footprint of a rover
Δt	Fixed simulation time step of the 3D engine

Chapter 1

Introduction

1.1 INTRODUCTION

Teams of inexpensive rovers are being considered for planetary survey, disaster-site inspection, agricultural monitoring and warehouse logistics, precisely the places where fixed communication infrastructure is absent or too costly to install. Such a team must organise itself: every few seconds someone has to decide which rover takes which task, and that decision is only as good as the information that reaches the decision maker. The information travels over the rovers' own radios, in this project low-power ESP-NOW links between ESP32-class microcontrollers. Each rover hears only a few nearby teammates, and every link drifts as rovers move, batteries drain and terrain gets in the way, so the communication network is a graph whose edges appear, weaken and disappear from one control cycle to the next.

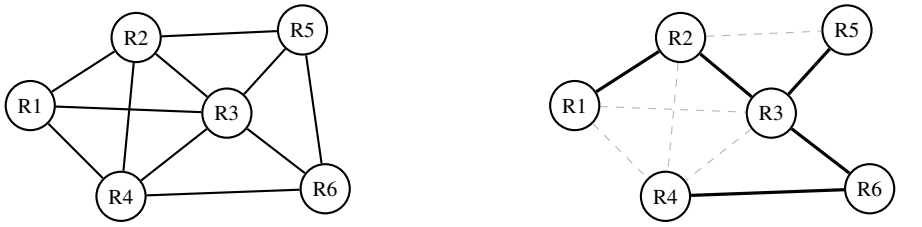
This report describes a complete pipeline for allocating tasks over that moving graph and, equally important, an audit of how well each part works. The pipeline first *prunes* the graph to a small, reliable working subgraph by solving a constrained optimisation problem written as a quadratic unconstrained binary optimisation (QUBO), the native input of quantum and quantum-inspired annealers. Once the graph has been pruned, a small graph neural network (GNN) gives every rover a score. The scores depend on trust values that the system learns online from how earlier tasks turned out, and a greedy step then hands out the tasks while respecting how much work each rover can take. We tested the pipeline in three settings, each more realistic than the last. All of the numbers in this report come from a statistical simulator; the other two settings are a physics simulation in Gazebo with ROS 2 and an interactive 3D simulation engine that we built for this project.

1.2 OVERVIEW OF DYNAMIC ROVER NETWORKS

We treat the fleet at control cycle t as an undirected graph $G_t = (V, E_t)$. Each vertex is a rover, and two rovers share an edge when they are able to exchange packets during that cycle. Each edge carries normalised features describing how much it is worth keeping (the trust and battery of its weaker endpoint, its link quality, proximity, freshness, the criticality of the tasks at its ends and its processing cost), and each rover carries its own trust, battery, load and capability. These features feed both stages: they decide which

edges survive pruning and how information flows between neighbours during scoring (Fig. 1.1).

The hardware follows a three-tier mist–fog–cloud split. The rovers (mist) sense and communicate; a gateway forwards telemetry to a Raspberry Pi (fog), which maintains the graph, prunes it and runs inference within the cycle deadline; and a laptop or server (cloud) hosts training and heavy optimisation. Compute is scarcest exactly where decisions must be made fastest, which shapes every design choice that follows.



(a) candidate graph: 11 links (b) pruned working graph: 5 links

Figure 1.1: The pruning idea: of the links in range (a), a small connected subset is kept (b, solid) subject to link caps, a cost budget and connectivity of the mission rovers.

1.3 CHALLENGES

1. **Volatile links.** A link can get better, get worse or vanish as rovers move, as interference changes and as distances grow. For this reason the model also has to track how recently each link was heard.
2. **Hidden, drifting reliability.** Nobody can measure how reliable a rover is. We can only estimate it from the tasks it was given, and which tasks it was given depended on how much we trusted it in the first place.
3. **Scarce fog compute.** Message passing costs grow with the number of edges, and the fog device has a small budget per cycle.
4. **Global constraints.** Whether the link caps, the processing budget and the connectivity of the mission rovers are respected depends on the kept graph as a whole. A rule that looks at one edge at a time has no way to guarantee any of them.
5. **Solver reliability.** Even a correctly encoded problem does not help if the solver gives back infeasible or weak answers, and this kind of failure is only noticed when the answers are compared with an exact solution.

1.4 PROJECT STATEMENT

To design, implement and evaluate a pipeline for dynamic rover networks that (i) prunes the communication graph every cycle with a constrained QUBO that enforces degree,

budget and connectivity constraints exactly; (ii) solves it reliably and checks the solver against exact classical ground truth; (iii) assigns tasks with a trust-aware GNN whose trust is learned online; (iv) demonstrates the pipeline in a physics simulator and an interactive real-time 3D engine; and (v) reports honestly where the architecture pays off and where simpler alternatives are as good or better.

1.5 OBJECTIVES

1. Formulate a constrained QUBO for dynamic graph pruning with exact, flow-based connectivity, and verify it against an independent re-derivation and exhaustive enumeration.
2. Explain why bit-level annealers fail on penalty-encoded QUBOs and build a solver that reliably returns feasible, near-optimal edge sets.
3. Set up exact classical baselines so that each claim about a solver can be checked against them.
4. Design a trust-weighted message-passing GNN with an online trust model, cheap enough for a fog device.
5. Build a causal closed-loop simulator with a statistically sound evaluation protocol, and a hostile automated test suite.
6. Deploy the pipeline in Gazebo with ROS 2 and in an interactive 3D simulation engine with live tuning and side-by-side comparisons.

1.6 SCOPE

The project covers the QUBO pruner, the trust-aware GNN and assignment, the three-tier architecture, the test suite, the Gazebo and ROS 2 integration and the 3D engine. Quantitative claims come from the statistical simulator, with 10 rovers for end-to-end experiments and 10 to 50 for scaling. Out of scope are experiments on physical rovers (a four-rover testbed is designed but not built), execution on quantum hardware (the QUBO matrix is exported but was not run on a quantum processor) and channel-accurate radio modelling.

1.7 CONTRIBUTIONS

1. A constrained QUBO for dynamic graph pruning in which connectivity is enforced exactly by a single-commodity flow encoding (Chapter 4).

2. A quantitative account of the *penalty barrier*: four serious annealing samplers find the exact optimum on only 48 to 64% of verifiable instances, while a feasibility-projected annealer finds it on all of them (Chapter 5).
3. Exact classical baselines showing that a direct mixed-integer program is exact and about 100 times faster, which fixes the QUBO's honest role as a hardware-target encoding (Chapter 6).
4. A trust-aware assignment layer that beats a trust-only baseline by 8.1 points of mission success with a confidence interval excluding zero (Chapters 7 and 10).
5. A causal evaluation with constraint-aware baselines, a ten-level constraint-pressure sweep, and an audit that caught a silent budget relaxation in the project's own earlier results (Chapters 8–10).
6. Two deployments: a containerised Gazebo Harmonic and ROS 2 Jazzy world (Chapter 11) and an interactive 3D engine with provably collision-free motion and a seven-draw-call renderer (Chapter 12).

1.8 ORGANIZATION OF THE REPORT

Chapter 2 surveys the background and related work, and Chapter 3 presents the system model. Chapters 4 to 6 form the optimisation core: formulation, solvers and exact baselines. Chapter 7 covers trust-aware inference and assignment, Chapter 8 the simulator and methodology, Chapter 9 verification, and Chapter 10 the results. Chapters 11 and 12 describe the Gazebo integration and the 3D engine, and Chapter 13 concludes.

Chapter 2

Background and Literature Survey

2.1 INTRODUCTION

This chapter introduces the foundations the project builds on, in three groups: learning (graph neural networks and trust), optimisation (QUBO models, annealing, tabu search and mixed-integer programming) and systems (the mist–fog–cloud split, robot middleware and real-time 3D rendering). It then surveys the closest related work and states the research gap.

2.2 GRAPH NEURAL NETWORKS

A graph neural network learns a representation of every node by repeatedly aggregating information from its neighbours. Kipf and Welling [15] popularised the graph convolutional network and Hamilton et al. [12] extended it to inductive settings. Most variants fit the *message-passing* template, in which node i combines its own state h_i with an aggregate of its neighbours' states:

$$m_i = \operatorname*{AGG}_{j \in \mathcal{N}(i)} h_j, \qquad h_i' = \phi(h_i, m_i), \tag{2.1}$$

with a permutation-invariant AGG and a learned update ϕ. For this project two properties of message passing are important. The cost of inference grows with the number of edges, so a pruned graph needs less work on the fog device. Also, if messages are weighted by the trust and quality of the link they travel over, a node can give less weight to neighbours that it cannot rely on.

2.3 QUBO MODELS AND THE PENALTY METHOD

A quadratic unconstrained binary optimisation problem asks for $z \in \{0,1\}^N$ minimising

$$H(z) = z^\top Q z + \text{const} \tag{2.2}$$

where Q is a symmetric matrix. Because $z_k^2 = z_k$ for binary variables, the linear terms can be stored on the diagonal. Writing $z_k = (1 + \sigma_k)/2$ with spins $\sigma_k \in \{-1, +1\}$ turns the problem into an Ising model, which is why the same problem can be given to classical, quantum-inspired and quantum annealers. Formulation techniques are sur-

veyed by Glover et al. [10] and by Lucas [21]. Since a QUBO cannot hold constraints directly, each constraint is turned into a penalty term. An equality $\sum_k a_k x_k = b$ becomes $\rho(\sum_k a_k x_k - b)^2$, and an inequality is first given a non-negative integer slack variable, written in binary. The weight ρ has to be large enough that breaking any constraint never pays off, and Chapter 5 shows what this safety costs.

2.4 ANNEALING, TABU SEARCH AND EXACT SOLVERS

Simulated annealing [16] works by proposing small changes. A change that raises the energy by ΔE is still accepted with probability $\exp(-\Delta E / \tau_{\mathrm{SA}})$, and the temperature τ_{SA} is slowly reduced during the run. D-Wave's Ocean software [6] includes *neal*, a production simulated annealing sampler. In quantum annealing [14], tunnelling driven by a transverse field takes the place of thermal fluctuations. Simulated quantum annealing imitates this process on a classical computer, and the OpenJij library [13] provides a sampler of this kind. Tabu search [9] is a local search method that escapes local minima by forbidding moves made recently. What these four methods have in common is that they change only a few bits of z at each step, and Chapter 5 shows that this is exactly what limits them.

A mixed-integer linear program (MILP) has a linear objective, linear constraints and some variables that must take integer values. Branch-and-cut solvers such as CBC [8] solve such programs exactly and prove that the answer is optimal. Requiring a set of nodes to be connected can be written compactly as a MILP using a *single-commodity flow*: a root node sends one unit of flow to every required node, and flow may only use selected edges. A standard result of network-flow theory [2] says that the flows are integral whenever the data are integral, so the flow variables can be left continuous.

2.5 FOG COMPUTING, TRUST AND TASK ALLOCATION

Bonomi et al. [5] introduced fog computing as a layer that sits between devices and the cloud, so that computation happens close to where the data is produced, and our deployment uses the same arrangement. Trust and reputation models judge how reliable an agent is from what it has done before, and Sabater and Sierra [24] review many such models. The simplest model that is still useful is an exponentially weighted moving average of outcomes. If trust is estimated without using the graph at all, picking the most trusted rover becomes a multi-armed bandit problem [4]. A trust-only baseline is therefore the strongest simple competitor, and we include it as a separate baseline. In the taxonomy of Korsah et al. [18], our setting has single-task robots, single-robot tasks and instantaneous assignment, recomputed every cycle.

2.6 ROBOT SIMULATION AND REAL-TIME RENDERING

ROS 2 [22] is the standard robot middleware: programs are *nodes* exchanging typed messages over named *topics*. Gazebo [17] is an open-source multi-robot simulator; its current generation, Gazebo Harmonic, describes worlds and robots in the Simulation Description Format (SDF), and a bridge forwards its state into ROS 2. In the browser, WebGL exposes the GPU and three.js [26] manages scenes on top of it. What matters most for performance is the number of *draw calls*, because every call has a fixed cost in the graphics driver. *Instanced rendering* draws many copies of the same geometry with a single call, and *geometry merging* joins parts that use the same material into one mesh. Chapter 12 uses both techniques so that the entire fleet is drawn with only a few calls.

2.7 SURVEY OF RELATED WORK

A number of papers prune the input graph of a GNN. STEP [19] trains a self-supervised pruner for large dynamic graphs, but it relies only on graph statistics and does not use telemetry like trust, link quality or battery level. CGP [20] co-sparsifies structure, features and weights during training, Wang et al. [28] score edges by their effect on the loss, and Peng et al. [23] sparsify GNN weights and structure; all target model or training efficiency on static benchmarks, and spectral sparsification [25] preserves spectra without hard constraints. On the allocation side, Ahanger et al. [1] use a quantum-inspired optimiser for IoT task allocation across edge and fog nodes, MAG-Q [27] combines graph attention, actor–critic learning and QUBO forwarding for task offloading in fog clusters, and Ali et al. [3] and STMTO [11] use trust for load balancing and UAV offloading. Table 2.1 summarises them.

Table 2.1: Qualitative comparison of related work with the present project.

Work	Main focus	Gap relative to this project
STEP [19]	unsupervised pruning of large dynamic graphs	no telemetry, trust or hard constraints
CGP [20]	gradual co-sparsification in training	training-time, static benchmarks
Wang et al. [28]	edge scoring by effect on the loss	model-side efficiency only
Peng et al. [23]	weight and structural sparsification	no adaptive physical-graph pruning
Ahanger et al. [1]	quantum-inspired IoT allocation	no trust-aware GNN pruning
MAG-Q [27]	GAT, actor–critic, QUBO forwarding	routing and placement, not pruning
Ali et al. [3]	trust-aware load balancing in MEC	trust is a scheduler input
STMTO [11]	trust-aware UAV offloading	no GNN-propagated trust or sparsification
This project	constrained QUBO pruning, trust GNN, solver audit, two simulators	physical testbed and quantum hardware pending

2.8 RESEARCH GAP

As far as we know, no earlier work combines constrained QUBO pruning with online trust-aware GNN inference for allocating tasks to rovers. We see three open gaps. First, existing pruning methods do not enforce hard combinatorial constraints. Second, trust-aware allocation methods do not spread trust through message passing. Third, papers that use QUBOs rarely check the solver's output against an exact solution, so a solver failure can easily be blamed on the formulation. This project works on all three gaps. It also compares the pipeline with the strongest classical alternative available, a step that many proposals leave out.

Chapter 3

System Model and Architecture

3.1 DESIGN GOALS

We fixed four goals for the architecture before choosing any algorithm. The first is that decisions have to respect the structure of the whole graph, since link caps, the budget and connectivity can only be judged on the full set of kept edges. The second is that the fog tier has to stay cheap, so everything that runs on the Raspberry Pi in every cycle should finish within a few milliseconds. The third is that learning has to feed back into the next cycle: trust learned from outcomes is used in the following cycle, and nothing inside one cycle depends on its own output. The last goal is that every claim should be checkable. Each stage can be tested against an independent reference, and mission success in the simulator depends causally on the decisions that are being evaluated.

3.2 THREE-TIER DEPLOYMENT

The deployment follows the three-tier split of Fig. 3.1. Rovers in the mist tier sense their state and exchange telemetry over ESP-NOW. A gateway forwards the telemetry over Wi-Fi to the fog tier, a Raspberry Pi that maintains the dynamic graph, prunes it, scores the rovers and assigns tasks. The cloud tier, a laptop or server reached over MQTT, retrains the GNN readout and runs the heavy optimisation experiments. Task assignments travel back down the same path.

The tier assignment is a design decision: rovers have too little compute for anything beyond sensing and radio, the fog tier must meet a per-cycle deadline, and the cloud has none. Section 10.5 quantifies a middle ground in which the cloud solves every few cycles and the fog device reuses the latest decision in between.

3.3 GRAPH MODEL AND FEATURES

At cycle t the fleet is the undirected graph $G_t = (V, E_t)$, with an edge whenever two rovers are within radio range. Table 3.1 lists the features attached to every edge and every rover. All continuous features are normalised to $[0, 1]$. Trust and battery on an edge take the minimum over the two endpoints, so a link is only as good as its weaker end, which is the conservative choice for communication reliability.

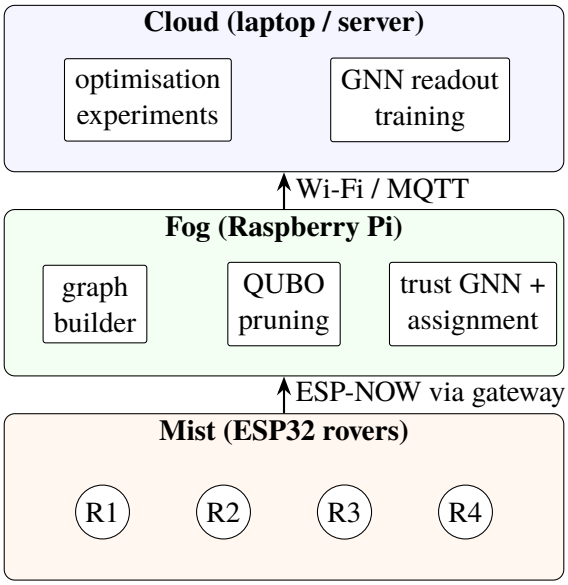


Figure 3.1: Three-tier deployment. Telemetry flows up and task assignments flow down; expensive solving and training run in the cloud or on the fog device depending on the cycle budget.

Table 3.1: Edge and rover features used by the pruner and the GNN.

Symbol	Name	Definition
T_e	edge trust	$\min(T_i^{t-1}, T_j^{t-1})$, from the previous cycle's trust model
L_e	link quality	measured quality of the last observation, with noise
B_e	edge battery	$\min(B_i, B_j)$
P_e	proximity	$1 - d_{ij}/d_{\max}$ for distance d_{ij} and radio range $d_{\max}$
A_e	freshness	$\exp(-\Delta t_e/\tau)$ since the last successful observation
C_e	task criticality	largest criticality of the tasks at the two endpoints
c_e	processing cost	$\max(1, \lfloor 10\,\bar{c}_e \rceil)$ integer budget units
T_i	rover trust	online EWMA estimate (Chapter 7)
B_i	battery	state of charge
ℓ_i	load	fraction of compute in use; $1 - \ell_i$ is available compute
q_i	capability	static hardware capability score

The raw processing cost gives the budget a physical meaning. Transmission time grows with distance and endpoint queues add processing time, so

$$\bar{c}_e = 0.1 + 0.5\,(1 - P_e) + 0.15\,(\ell_i + \ell_j), \tag{3.1}$$

and the integer cost c_e lies between 1 and 10. Because every edge costs at least one unit, any spanning tree on n rovers costs at least $n - 1$ units; the 3D engine of Chapter 12 uses this bound to warn when a chosen budget can never be met.

The usefulness of an edge is summarised by a weighted utility

$$U_e = 0.25\,T_e + 0.25\,L_e + 0.10\,B_e + 0.10\,P_e + 0.15\,A_e + 0.15\,C_e, \tag{3.2}$$

whose weights sum to one, so $U_e \in [0, 1]$. The same utility is used by every pruning baseline, which keeps comparisons between pruners fair.

3.4 ONE PIPELINE CYCLE

Each control cycle executes the pipeline of Fig. 3.2. The graph builder turns telemetry into G_t with fresh features. Next, the pruner picks a working subgraph that meets the constraints, and the GNN uses that pruned graph to score each rover for each task that is waiting. The assignment stage then matches tasks to rovers greedily, without giving any rover more than it can handle. At the end of the cycle we see which tasks succeeded and update the trust values.

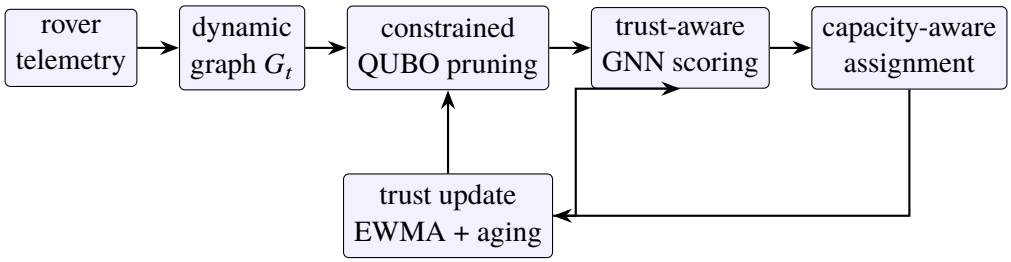


Figure 3.2: One pipeline cycle. The trust values from the previous cycle are used in the pruner's edge features and in the GNN's node features, so the loop is closed across cycles and no stage depends on its own output within a cycle.

3.5 THE MISSION MODEL

Pruning the graph only makes sense if the choice of kept links can actually change how a mission turns out. For this reason the mission model treats a successful task as two linked events.

Definition 3.1 (Execution and delivery)**.** When rover i is given a task, it *executes* the task with a probability equal to its hidden reliability, as long as its battery is above 15%.

After that, the result has to be *delivered* to the root rover through the kept subgraph. Delivery succeeds with a probability given by the delivery factor

$$p_i = \max_{\pi : i \to r} \min_{e \in \pi} L_e,$$

which is the quality of the weakest link on the *widest* path π from i to the root r in the kept graph. By definition $p_r = 1$, and $p_i = 0$ when no path exists. The task only counts as a success if both events happen.

Trust is updated from execution outcomes only, which means it is an estimate of reliability. Delivery depends on the kept edges, so every pruning decision has a real, measurable effect on mission success. An earlier version of the simulator lacked this coupling: success depended on rover reliability alone and the whole pruning layer was causally disconnected from the metric it was meant to improve. Restoring the coupling was a precondition for every end-to-end claim in Chapter 10.

3.6 SOFTWARE ORGANISATION

The software mirrors the stages of Fig. 3.2 and adds the verification and simulation layers around them. Table 3.2 lists the components by responsibility; they are written in Python with NumPy, except the 3D engine, which is written in TypeScript.

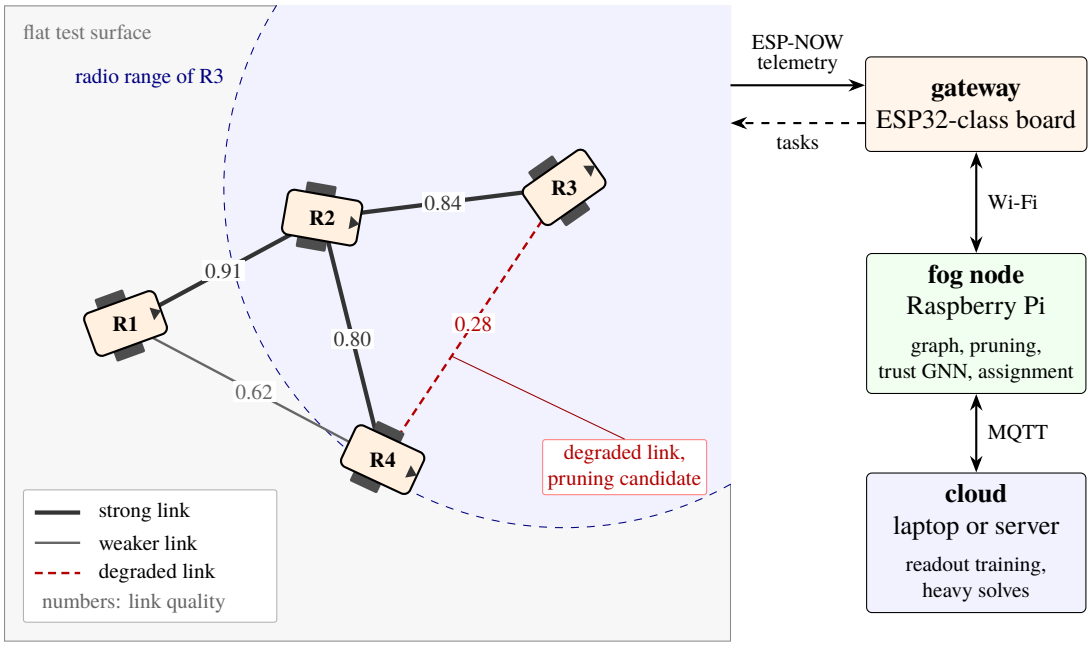
Table 3.2: Software components and their responsibilities.

Component	Responsibility
QUBO builder	Assembles the Q matrix with slack and flow variables, checks inputs, repairs edge sets to feasibility and exports the matrix
Solvers	Projected edge annealing (production) and bit-level simulated annealing (baseline)
Exact baselines	Direct MILP of the constrained problem and an exact linearised solve of the Q matrix, both with CBC
Trust GNN	Two-layer trust-weighted message passing, gating and scoring
Trust model and assignment	EWMA trust with aging; capacity-aware greedy assignment; readout training
Closed-loop simulator	Mobility, radio, hidden reliability, faults, pruner and assigner arms, metrics and statistics
Experiment drivers	Scaling runs, a 16-seed pruner benchmark and the constraint-pressure sweeps
Verification suite	Roughly five hundred automated checks in fourteen sections
Gazebo planner node	Runs the pipeline against live odometry inside ROS 2
Planner service and 3D engine	Live sessions over HTTP and the interactive browser simulation

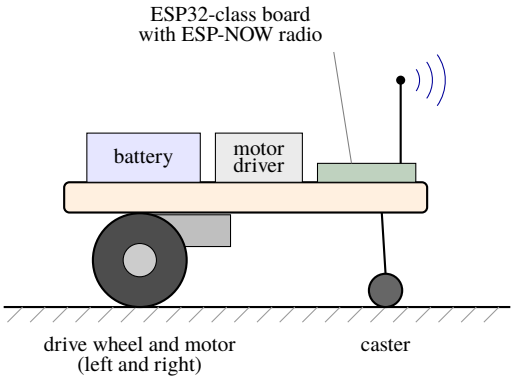
3.7 PHYSICAL TESTBED CONCEPT

The intended physical testbed is four rovers on a flat surface, each an ESP32-class board on a small differential-drive chassis, with ESP-NOW links whose quality varies with distance (Fig. 3.3). A degraded link, such as the one between R3 and R4 in panel (a), is exactly the kind of edge the pruner is meant to drop; panel (c) shows one kept graph without it, in which all four rovers remain connected. The testbed has not yet been built; all quantitative results in this report come from simulation, and the Gazebo world of Chapter 11 reproduces the same four-rover line-up in a physics engine.

(a) Test surface and data path, top view



(b) Rover concept, side view, and the data it exchanges



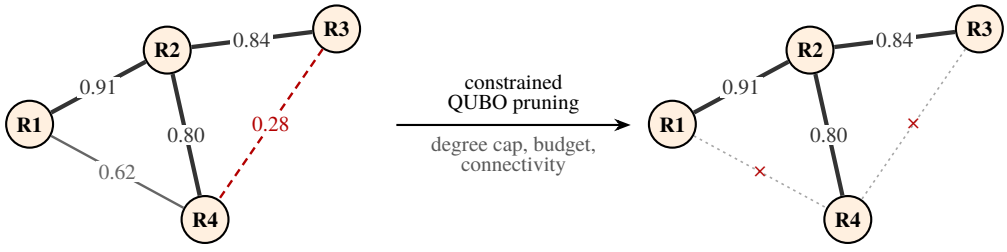
Each cycle, rover to fog node

- position and heading
- battery level
- compute load
- link quality to each neighbour (signal strength, packet loss)
- outcome of the last assigned task

Fog node to rover

- task assignment

(c) Pruning the candidate graph



candidate graph: 5 links

kept graph: 3 links, all rovers connected

Figure 3.3: Physical testbed concept. (a) Four ESP32-class rovers on a flat surface; line width shows ESP-NOW link quality (illustrative values), and the link from R3 to R4, near the edge of R3's radio range, is degraded. An ESP32 gateway relays telemetry to the Raspberry Pi fog node and returns task assignments. (b) One rover in side view and the data it exchanges each cycle. (c) The candidate graph of (a) and one possible kept graph: the degraded link and a redundant link are pruned while all four rovers stay connected.

Chapter 4

Constrained QUBO Formulation

4.1 OVERVIEW

The pruning stage chooses, in every cycle, which edges of G_t to keep. The choice is combinatorial: it must maximise the usefulness of the kept edges while respecting per-rover degree caps, a global cost budget and the connectivity of a set of required rovers. This chapter writes that problem as a single QUBO, explains every term, proves the two properties that make the encoding correct, works through a five-rover example by hand and reports how the implementation was verified.

4.2 DECISION VARIABLES AND NOTATION

Let $H \subseteq E_t$ be the *hard-critical* edges, which are always kept (for example a relay rover's only link), and $O = E_t \setminus H$ the optional edges. Let R be the required rovers with root $r \in R$ and $m = |R| - 1$, let D_i be the degree cap of rover i, and let K be the global budget in integer cost units. Writing $\deg_H(i)$ for the number of hard edges at i and c_H for their total cost, the remaining degree capacity of rover i is $D_i - \deg_H(i)$ and the remaining budget is $K - c_H$. Table 4.1 lists the four families of binary variables.

Table 4.1: Variable families of the QUBO. Every integer is binary-encoded.

Variable	Meaning	Number of bits
$x_e,\ e \in O$	keep (1) or prune (0) optional edge e	$\lvert O\rvert$
$s_i,\ i \in V$	degree slack, $0 \le s_i \le D_i - \deg_H(i)$	$\sum_i \lceil \log_2(D_i - \deg_H(i) + 1) \rceil$
s_C	budget slack, $0 \le s_C \le K - c_H$	$\lceil \log_2(K - c_H + 1) \rceil$
f_{uv},$f_{vu}, q_e,\ e \in E_t$	two directed flows and a capacity slack per edge, each in $[0, m]$	$3\lvert E_t\rvert \lceil \log_2(m + 1) \rceil$

For notational convenience $x_e := 1$ for $e \in H$. The vector z stacks all bits, and N denotes its length.

4.3 THE ENERGY FUNCTION

The complete energy is

$$
\begin{aligned}
H(z) = \; & \sum_{e \in O} (-U_e + \lambda)\, x_e + \eta \sum_{e \in O} \left(x_e - x_e^{t-1}\right)^2 \\
& + \rho_d \sum_{i \in V} \Big(\sum_{e \in \delta_O(i)} x_e + s_i - \left(D_i - \deg_H(i)\right) \Big)^2 \\
& + \rho_C \Big(\sum_{e \in O} c_e x_e + s_C - \left(K - c_H\right) \Big)^2 \\
& + \rho_F \sum_{v \in V} \left(\mathrm{in}_v - \mathrm{out}_v - b_v\right)^2 \\
& + \rho_{FC} \sum_{e=(u,v) \in E_t} \left(f_{uv} + f_{vu} + q_e - m\, x_e\right)^2, \qquad (4.1)
\end{aligned}
$$

where $\delta_O(i)$ is the set of optional edges at rover i; the total flows entering and leaving rover v are in_v and out_v; and the supplies are $b_r = -m$, $b_v = 1$ for $v \in R \setminus \{r\}$ and $b_v = 0$ otherwise. Each squared bracket is a polynomial of degree two in the bits, so the whole expression expands into the form $z^\top Q z + \text{const}$. Table 4.2 collects the constants.

Table 4.2: Constants of the QUBO used throughout the report.

Constant	Role	Value
λ	sparsity penalty per kept edge	0.08
η	stability penalty per changed decision	0.15
ρ_d, ρ_C	degree and budget penalty weights	100
ρ_F, ρ_{FC}	flow conservation and flow capacity weights	500
cost scale	quantisation of $\bar{c}_e$ into budget units	10
utility weights	trust, link, battery, proximity, freshness, task	0.25, 0.25, 0.10, 0.10, 0.15, 0.15

4.4 THE SOFT TERMS

The coefficient $-U_e + \lambda$ rewards an edge for its utility and charges a constant λ for keeping it, so edges with $U_e < \lambda$ are never worth keeping alone and the constant pressure drives solutions towards sparse graphs. The stability term charges η for every decision that changes relative to the previous cycle; since $x_e^2 = x_e$ it is linear, $\eta(x_e(1 - 2x_e^{t-1}) + x_e^{t-1})$, and it stops the kept topology from flapping between nearly equal alternatives, which would confuse both the GNN and the trust model.

4.5 ENCODING INEQUALITIES WITH SLACK

Degree caps and the budget are inequalities. The standard device turns $\sum_k a_k x_k \le B$ into the equality $\sum_k a_k x_k + s = B$ with an integer slack $0 \le s \le B$, writes $s = \sum_j 2^j s_j$ in bits, and penalises the squared residual. Expanding a single squared constraint $(a^\top y - B)^2$ over a bit vector y and using $y_k^2 = y_k$ gives

$$(a^\top y - B)^2 = \sum_k (a_k^2 - 2Ba_k) y_k + 2 \sum_{k<l} a_k a_l y_k y_l + B^2, \tag{4.2}$$

so a constraint contributes $a_k^2 - 2Ba_k$ to the diagonal entry of each of its bits, $a_k a_l$ to each symmetric off-diagonal pair, and B^2 to the constant.

Example 4.1 (One degree constraint)**.** Take a rover with cap $D_i = 2$, no hard edges and three optional edges x_1, x_2, x_3. The slack $s_i \in \{0, 1, 2, 3\}$ needs two bits, $s_i = s_0 + 2s_1$, so the coefficient vector over $(x_1, x_2, x_3, s_0, s_1)$ is $a = (1, 1, 1, 1, 2)$ and $B = 2$. Equation (4.2) yields the entries of Table 4.3, all multiplied by ρ_d.

Table 4.3: Coefficients generated by one degree constraint with $D_i = 2$ (before multiplication by ρ_d).

Bit	Linear term $a_k^2 - 2Ba_k$	Pairwise term $2a_k a_l$
x_1, x_2, x_3	-3 each	2 with each other and with s_0; 4 with s_1
s_0	-3	2 with each x; 4 with s_1
s_1	-4	4 with each x and with s_0
constant	$B^2 = 4$	

Three checks confirm the intended behaviour. Keeping two edges with both slack bits zero gives $(2 - 2)^2 = 0$. Keeping one edge with $s_0 = 1$ also gives 0. Keeping all three edges gives at least $(3 - 2)^2 = 1$ for every slack setting, because slack cannot be negative; the violation therefore costs at least $\rho_d = 100$, while the third edge can gain at most about 1.2.

4.6 THE BUDGET CONSTRAINT

The budget term has the same structure, with the integer costs c_e as coefficients and $K - c_H$ as right-hand side, and its slack needs only $\lceil \log_2(K - c_H + 1) \rceil$ bits. A budget below the cost of every connected subgraph is *infeasible* together with connectivity, and no penalty weighting can change that; Section 5.7 shows how the production solver detects and records this case, and Section 10.2.1 shows that the project's own earlier experiments fell into exactly this trap.

4.7 EXACT CONNECTIVITY BY SINGLE-COMMODITY FLOW

The two flow terms enforce connectivity of the required set inside the optimisation itself, rather than through a check applied afterwards.

Proposition 4.1 (Flow encodes connectivity)**.** *Let x be a selection of edges. There exist integer flows and slacks that make both flow terms of Eq.* (4.1) *zero if and only if every required rover is connected to the root r in the subgraph of selected edges.*

Proof. ($\Leftarrow$) Suppose every required rover is connected to r. Take a spanning tree of the component that contains the root, and send one unit from r to each required rover along the single path the tree gives. Every arc then carries at most m units, conservation holds at every node with the given supplies, and each selected edge satisfies $f_{uv} + f_{vu} \leq m$, so a slack $q_e \in [0, m]$ closes the capacity equality. Unselected edges carry no flow and take $q_e = 0$. ($\Rightarrow$) If both terms are zero, conservation says that m units leave r and one unit is absorbed at each other required rover, while capacity forces $f_{uv} = f_{vu} = 0$ on every unselected edge. Decomposing the flow into paths, each required rover receives its unit along a path of selected edges from r, so it is connected to r. □

4.8 WHY THE PENALTIES DOMINATE

Proposition 4.2 (No violation is profitable)**.** *With $\rho_d = \rho_C = 100$, $\rho_F = \rho_{FC} = 500$ and soft coefficients of magnitude at most about* 1.2 *per edge, every exact minimiser of Eq.* (4.1) *satisfies all constraints whenever a feasible selection exists.*

Argument. A violated degree or budget constraint contributes at least $\rho = 100$, because its residual is a non-zero integer and enters squared, while each unit of violation unlocks at most one edge worth less than $1 + \lambda + \eta < 1.25$. Flow violations are penalised quadratically against an at most linear gain. □

The argument is confirmed exhaustively in Section 4.10. It also explains the difficulty of Chapter 5: the same walls that make violations unprofitable make them unreachable for a sampler that must climb over them.

4.9 A WORKED EXAMPLE ON FIVE ROVERS

The whole formulation is visible on an instance small enough to solve by hand. There are five rovers: the root 0 and rovers 1 to 4. Seven links are in range. The link 0–1 is hard-critical, being rover 1's only tether. Rovers $\{0, 2, 3, 4\}$ are required, so rover 1 is a relay that may be cut off. The degree caps are $(3, 2, 3, 2, 2)$ for rovers 0 to 4 and the

budget is $K = 12$ units. Table 4.4 gives the utility, the integer cost and the previous decision of every link.

Table 4.4: The five-rover worked example: utility U_e from Eq. (3.2), integer cost c_e and previous-cycle decision x_e^{t-1}.

Link	U_e	c_e	x_e^{t-1}
0–1 (hard)	0.810	2	1
0–2	0.900	1	1
0–3	0.405	8	0
1–2	0.705	3	1
2–3	0.750	4	1
2–4	0.790	2	0
3–4	0.555	6	1

Size. There are six optional edges. With $m = 3$ every flow integer needs two bits, so the flow family has $3 \times 7 \times 2 = 42$ bits; the degree slacks need $2+1+2+2+2 = 9$ bits and the budget slack, with $K - c_H = 10$, needs 4. The QUBO therefore has $6 + 42 + 9 + 4 = 61$ binary variables, even though the decision itself has only $2^6 = 64$ possibilities.

Enumeration. All 64 optional-edge subsets were enumerated. Only **3** are feasible. The optimum keeps $\{0\text{–}1, 0\text{–}2, 2\text{–}3, 2\text{–}4\}$ at a cost of 9 units, with energy

$$\underbrace{(-0.82) + (-0.67) + (-0.71)}_{\text{utility and sparsity}} + \underbrace{0.15 \times 3}_{\text{stability}} = -1.750,$$

where the three stability charges come from dropping 1–2 and 3–4 and adding 2–4. The other two feasible subsets both score -1.555.

The trap. Thirty of the 61 infeasible subsets violate only connectivity. The best one among them is $\{0\text{–}1, 0\text{–}2, 1\text{–}2, 2\text{–}3\}$. Its soft energy, -1.965, is actually *lower* than the optimum's, because it keeps the attractive link 1–2 and pays just one stability charge. The problem is that rover 4 is no longer connected to the root. A pruner that only looked at soft scores, or that checked connectivity after choosing the edges, would pick this subset, since it looks better by 0.215. In the QUBO, however, the flow terms add at least 500 to it, which puts its real energy above +498. The cheapest subset that breaks a degree or budget constraint has energy +97.475. The gap between that subset and the optimum is 99.2, nearly a hundred times wider than the whole range of feasible energies ($[-1.750, -1.555]$). Fig. 4.1 shows these energy levels.

Cross-checks. We also computed the optimum's energy from the matrix, as $z^\top Q z +$ const on the feasible bit vector. It comes out at -1.750, the same as enumeration. The production annealer of Chapter 5 finds the same four edges without any violation, and

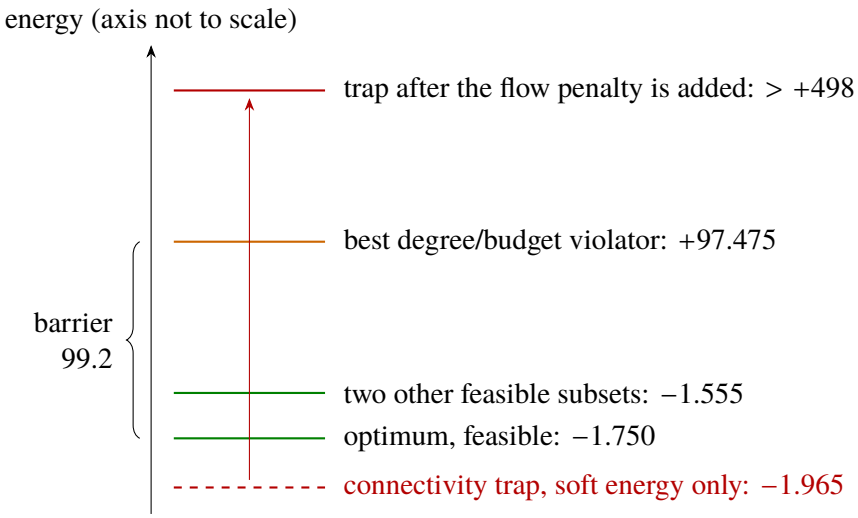


Figure 4.1: Energy ladder of the worked example. If connectivity is ignored, the trap seems better than the optimum. Once the flow penalty is included, it ends up far above all of the feasible subsets.

the exact MILP of Chapter 6 reaches the same soft energy, so all three methods give the same answer.

4.10 VERIFICATION OF THE ENCODING

We ran two more checks, this time on randomly generated instances.

1. **Exhaustive optimum check.** On 38 random instances we found the exact bit-level optimum of Eq. (4.1). In none of them did the optimum break a constraint, which is what the dominance argument says should happen.

2. **Independent re-derivation.** A second piece of code computes the energy straight from the specification, term by term, without building Q. We compared the two on 40 instances with 8 random states each, and the matrix energy matched this independent value to within 2.3×10^{-10}.

These checks give strong empirical support that the implementation is right, but they are not a formal proof. What they do settle is a question that comes up again and again in the next chapter: if a solver later returns a poor answer, the solver is to blame and not the encoding.

4.11 PROBLEM SIZE

The flow family dominates the variable count, and the matrix is stored densely, so memory grows with the square of the number of bits. On range-limited random fleets with connectivity enforced, ten rovers give 427 binary variables (a 1.5 MB matrix) and twenty

rovers 1076 variables (9.3 MB); without connectivity, fifty rovers need only 328. The dense formulation therefore targets tens of rovers rather than hundreds, a limit revisited in Chapter 13.

Chapter 5

Solving the QUBO: the Penalty Barrier and Projected Annealing

5.1 ANNEALING A PENALTY-ENCODED QUBO

A QUBO such as Eq. (4.1) is written to be sampled by an annealer, classical or quantum. The natural first solver is therefore a bit-level simulated annealer that flips one bit of z at a time and accepts uphill moves with probability $\exp(-\Delta E/\tau_{\mathrm{SA}})$. For a symmetric Q the energy change of flipping bit k is cheap to compute,

$$\Delta E_k = 2\,(1 - 2z_k)\,(Qz)_k + Q_{kk}, \tag{5.1}$$

and maintaining the vector Qz incrementally makes each proposal $O(N)$. The experiments of this chapter show that this solver, and three serious alternatives, fail on our problem in a specific and quantifiable way, even though the encoding was verified to be correct.

5.2 THE PENALTY BARRIER

Consider any feasible state. Every single-bit flip either keeps the state feasible, changing the soft objective by at most about 1.2, or breaks a constraint, raising the energy by at least 100. A flip of an edge bit that keeps feasibility is rare, because the slack and flow bits must change together with it; most single flips therefore break something. The probability of accepting a constraint-breaking move at temperature τ_{SA} is

$$\Pr[\text{accept}] = e^{-100/\tau_{\mathrm{SA}}}, \qquad e^{-100} \approx 3.7 \times 10^{-44}, \qquad e^{-10} \approx 4.5 \times 10^{-5}.$$

At $\tau_{\mathrm{SA}} = 1$, a temperature already hot for a soft landscape of width about one, the acceptance probability is astronomically small; even at $\tau_{\mathrm{SA}} = 10$ it is only 4.5×10^{-5} per attempt, and at that temperature the soft landscape is pure noise. A bit-level annealer therefore cannot travel between feasible regions: it either freezes in the basin of its starting state or wanders through infeasible space and returns an infeasible answer. Fig. 5.1 sketches the situation.

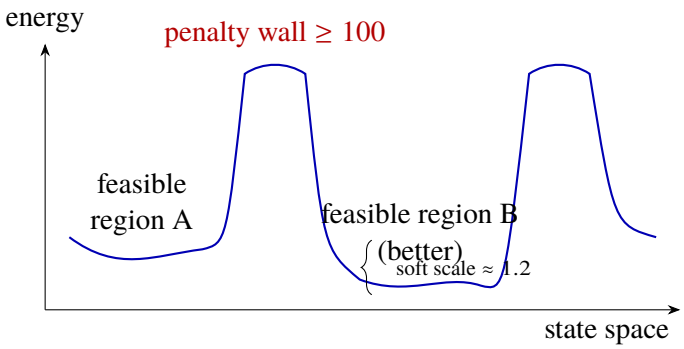


Figure 5.1: Schematic (not to scale) of the penalty barrier. Moves inside a basin change the energy by about one unit, but crossing from feasible region A to the better region B needs a climb of at least 100.

5.3 BENCHMARKING FOUR SERIOUS SAMPLERS

To rule out a weak implementation as the explanation, the actual Q matrices of our instances were given to four annealing-family solvers. The benchmark uses 25 small instances, each with four to seven rovers and no more than sixteen optional edges. We generated them with tight degree and budget caps, so the constraints actually limit the answer. The ground truth H^* of every instance is the exact optimum of the linearised Q matrix solved by CBC (Chapter 6), cross-checked against exhaustive enumeration of all edge subsets on instances with at most 60 QUBO variables. Each sampler's answer is projected to the feasible edge set it selects before scoring, which favours the samplers. The samplers were bit-flip SA (20 restarts, cooling 0.98, feasible warm start), *neal* (100 reads), tabu search (10 reads, 200 ms limit) and OpenJij SQA (25 reads); Fig. 5.2 plots their exact-solve rates.

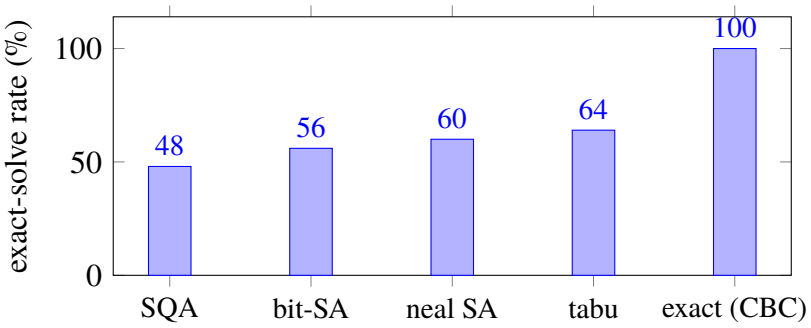


Figure 5.2: Exact-solve rate of four annealing-family samplers on the actual Q matrices of 25 instances. The exact linearised solve defines the optimum.

The difficulty is not specific to one implementation or one move set: all four samplers miss at least a third of the instances, including the production *neal* sampler, a deterministic tabu search and a quantum-inspired sampler. Random restarts of the bit-level annealer, without a feasible warm start, returned *infeasible* states in 19 of the 25 instances: the samplers are not converging to slightly wrong optima, they are failing to

find legal states at all. The claim is deliberately narrow. On this penalty-encoded landscape, annealing-family QUBO samplers are unreliable; the same matrices are solved exactly, without difficulty, by an exact method.

5.4 PROJECTED EDGE ANNEALING

The production solver developed in this project, *projected edge annealing*, keeps the QUBO as the specification of the problem but changes the space in which the search moves: it moves between feasible *edge sets* instead of between bit strings.

Moves. A move either toggles one optional edge or swaps two edges (one in, one out), each with probability one half. Swaps matter because many improvements, such as replacing one tree edge by a better one, pass through an infeasible intermediate state when done one edge at a time.

Projection. Every candidate edge set is projected back onto the feasible manifold by an exact repair procedure: degree and budget slacks are computed in closed form from the selected edges, and a spanning-tree flow from the root to every required rover is recomputed. Sometimes no feasible completion exists, for example when the candidate cuts off a required rover, and in that case the move is simply rejected.

Objective. In a feasible state all of the penalty terms are zero, so the energy is just the soft objective, and computing it takes $O(|O|)$ time. This means the solver is still minimising exactly $H(z)$, only over the feasible states, and the matrix Q can still be exported to annealing hardware.

Seeding and restarts. Each restart begins from a random spanning tree that respects the degree caps. We do not grow the starting point greedily, one edge at a time. With connectivity constraints, every partial version of a feasible tree may itself be infeasible, and a greedy builder can get stuck before it ever reaches a valid state. The first restart can instead begin from a *warm* edge set, which is the previous cycle's decision restricted to the links that still exist.

Polish. When the temperature schedule is over, a deterministic polish step takes over. It switches between passes of single toggles and passes of pairwise swaps, and stops once no single move lowers the energy any further.

Since every candidate is feasible by construction, the search never meets the penalty walls. It only has to move around the soft landscape, where energy differences are of order one. In this sense projection *removes* the barrier instead of tuning the solver to get around it.

For the benchmark, the schedule cools from temperature 1.0 to 0.01 at rate 0.998, with four steps at each temperature and five restarts. Inside the closed loop we use a faster schedule (rate 0.995, two steps, two restarts), and the warm start counts as one of the restarts. Appendix 1 gives the pseudo-code.

5.5 HOW GOOD IS THE PROJECTED ANNEALER?

We left the projected annealer *out* of the sampler comparison above on purpose. Its moves do more than flip single bits, so putting it next to the samplers would not be a fair comparison. Its own performance is reported separately, as an engineering result.

1. On the 25 exactly verifiable instances it finds the exact optimum in **100%** of cases, using about 2.4×10^4 energy evaluations and 2.1 s per solve.
2. On 20 larger instances with up to 40 optional edges, checked against exact MILP optima, it is exact in **95%** of cases, with a mean normalised gap of 7×10^{-5} and a worst case of 1.4×10^{-3}.
3. On 20 instances with up to 817 binary variables, where no exact optimum was computed, its gap against the best of four annealer seeds is 6.9% on average (90th percentile 17%), which measures consistency across seeds rather than distance from the optimum.

Throughout, the normalised gap is

$$\text{gap} = \frac{H_{\text{SA}} - H^*}{\max(1, |H^*|)}, \tag{5.2}$$

where the $\max(1, \cdot)$ prevents a near-zero optimum from inflating the ratio.

5.6 SENSITIVITY TO THE ENCODING'S FREE PARAMETERS

The utility weights and penalties are chosen by hand, so a fair question is whether the solver works only for one lucky setting. On a fixed instance set the projected annealer is exact on 87% of instances, and it stays at exactly 87% when the trust weight is scaled by 1.5, when the utility weights are made uniform, when all penalties are divided or multiplied by five, when the sparsity penalty is raised to 0.3 and when the stability penalty is raised to 0.6. The solver is not fragile to the encoding's free parameters.

5.7 THE RELAXATION LADDER

In a live system the constraints can be jointly infeasible: a budget below the cheapest connected subgraph, or a degree cap of two on a radio graph that has no chain through

every required rover. The production pruner therefore tries a ladder of progressively relaxed problems and records which rung it used in every cycle (Table 5.1).

Table 5.1: The relaxation ladder of the production pruner, tried top to bottom.

Rung	Constraints enforced
full	degree caps, budget and connectivity of the reachable required rovers
no-budget	degree caps and connectivity; budget dropped
budget-only	degree caps and budget; connectivity dropped
unconstrained	degree caps only
keep-all fallback	nothing could be satisfied; every candidate link is kept

Two details make the ladder honest. First, the required set is the fixed mission roster restricted to the rovers currently reachable from the root; unreachable mission rovers are counted, not silently redefined away. Second, the rung used is logged every cycle, so the coverage of each guarantee is a measured number rather than an assertion. That bookkeeping later exposed the silent budget relaxation of Section 10.2.1.

Remark 5.1 (A restart floor). The warm start counts as one of the solver's restarts. With a single restart and an infeasible warm start, such as the very first cycle when every candidate link is still marked as kept, no random spanning tree is ever tried, so a perfectly solvable instance falls through every rung of the ladder to the keep-all fallback. The defect was found while calibrating the 3D engine of Chapter 12, whose minimum number of restarts is therefore two.

5.8 IMPLICATIONS

The central finding is that a correct encoding is necessary but not sufficient. A generic bit-level annealer can fail to recover the optimum of a correct QUBO because of the very penalty walls the encoding needs, and that failure is invisible unless solver output is checked against exact ground truth. Moving the search into the feasible space removes the barrier for the classical solver. Whether quantum annealing hardware, whose transverse-field dynamics differ from thermal sampling, behaves better on this landscape is an open question that the exported Q matrices are ready to answer.

Chapter 6

Exact Classical Baselines

6.1 WHY AN EXACT BASELINE

Every solver claim in Chapter 5 needs a referee. Exhaustive enumeration is a perfect referee but only for a dozen or so optional edges, because the number of subsets doubles with each edge. The fairest question to ask of any QUBO pipeline is also a sharper one: why not solve the original constrained problem directly with the best classical tool available? This chapter builds two exact baselines that answer both needs, using the open-source branch-and-cut solver CBC [8].

1. A **direct MILP** of the constrained pruning problem, which never forms the penalty encoding at all.
2. An **exact linearised solve** of the Q matrix itself, which certifies the true minimum of exactly the object the samplers were given.

6.2 THE DIRECT MIXED-INTEGER PROGRAM

The soft part of Eq. (4.1) is linear in x: the utility term is linear, and the stability term is linear for binary variables because $(x_e - x_e^{t-1})^2 = x_e(1 - 2x_e^{t-1}) + x_e^{t-1}$. The constraints are linear too, so the whole problem is a mixed-integer linear program:

$$\min_{x,f} \quad \sum_{e\in O} \Big(-U_e + \lambda + \eta\,(1 - 2x_e^{t-1})\Big)x_e \tag{6.1}$$

$$\text{s.t.} \quad \sum_{e\in\delta_O(i)} x_e \le D_i - \deg_H(i) \qquad \forall i \in V, \tag{6.2}$$

$$\sum_{e\in O} c_e x_e \le K - c_H, \tag{6.3}$$

$$\text{in}_v - \text{out}_v = b_v \qquad \forall v \in V, \tag{6.4}$$

$$f_{uv} + f_{vu} \le m\,x_e \qquad \forall e = (u, v) \in E_t, \tag{6.5}$$

$$x_e \in \{0, 1\}, \quad f_{uv} \ge 0 \text{ (continuous)}.$$

Only the edge decisions are integer. The flows are continuous, which keeps the branch-and-bound tree small, and the next proposition shows that nothing is lost by relaxing them.

Proposition 6.1 (Flow integrality)**.** *For any fixed binary x, the flow constraints define a network-flow polytope with integer supplies and integer capacities. If it is non-empty, it has an integral point.*

Argument. With x fixed, the constraints are those of a single-commodity flow on the selected edges with supplies $b_v \in \{-m, 0, 1\}$ and capacities $m\,x_e$. The constraint matrix of a network flow is totally unimodular, so every vertex of the polytope is integral [2]. Hence a continuous feasible flow exists exactly when an integer one does, and the MILP optimum equals the optimum of the fully integer problem. □

Because this program never contains a penalty, it has no barriers: the solver reasons about feasibility directly, through linear-programming bounds and cuts.

6.3 THE EXACT LINEARISED SOLVE OF THE Q MATRIX

The second baseline answers a different question: what is the true minimum of the very matrix Q that the samplers sampled? A quadratic binary objective becomes linear through the standard product linearisation. For every non-zero off-diagonal pair Q_{kl} with $k < l$ we introduce a variable y_{kl} intended to equal $z_k z_l$, and enforce

$$y_{kl} \le z_k, \qquad y_{kl} \le z_l, \qquad y_{kl} \ge z_k + z_l - 1, \qquad y_{kl} \ge 0. \tag{6.6}$$

For binary z_k, z_l these inequalities force $y_{kl} = z_k z_l$ exactly. The objective becomes

$$\sum_k Q_{kk} z_k + 2 \sum_{k<l} Q_{kl}\, y_{kl} + \text{const}, \tag{6.7}$$

and CBC minimises it. An answer is only used when CBC reports that it has proven optimality. If the solver stops for some other reason, for example because it hit the time limit, that instance is left out instead of being scored. One auxiliary variable per non-zero pair makes the program grow quadratically, so this baseline is used for matrices with at most about 60 variables, where it serves as the ground truth of the sampler benchmark.

6.4 CROSS-VALIDATION OF THE REFEREES

A referee must itself be checked. Both baselines were compared against exhaustive enumeration on small instances, where all three must agree:

- the direct MILP optimum equals the best feasible edge set found by enumerating every subset of optional edges;

- the linearised Q-matrix optimum equals the same energy, confirming again, from a third direction, that the penalty encoding has no spurious infeasible minimum;
- on the five-rover worked example of Section 4.9, the MILP returns exactly the four optimal edges with soft energy -1.750.

After the methods agreed on the small instances, we used the direct MILP as the reference for the larger ones, where enumerating every subset is not possible.

6.5 RESULTS

In Table 6.1 the projected annealer is compared with the exact MILP on 20 instances that have up to 40 optional edges. Table 6.2 returns to the scaling study and adds the MILP optimum for the sizes where we could compute it.

Table 6.1: Projected annealer and exact MILP compared on 20 instances.

Measure	Projected annealer	MILP (CBC)
Share solved exactly	95%	100% (by definition)
Average normalised gap	7×10^{-5}	0
Largest normalised gap	1.4×10^{-3}	0
Time per instance	about 2.1 s	about 11 to 15 ms
Proves optimality	no	yes

Table 6.2: Soft energy on the scaling instances (lower values are better) for the projected annealer, a greedy top-K baseline that is repaired to feasibility, and the exact MILP where we could run it. Connectivity is only enforced up to $n = 20$, and at $n = 10$ the repaired greedy edge set is still infeasible.

n	Projected	Greedy (repaired)	MILP
10	-6.39	infeasible	-6.39
20	-15.05	-11.76	-15.36
30	-21.12	-16.62	not run
50	-36.71	-24.30	not run

We take two things from these tables. The first is that the projected annealer comes very close to the exact answer. It matches the optimum at ten rovers and is within about 2% of it at twenty. It is also well ahead of the repaired greedy baseline at every size, and the lead gets bigger as the fleet grows (3.3 energy units at 20 rovers and 12.4 at 50). The second point is less comfortable for anyone who wants to put the QUBO first: the MILP is exact *and* about 100 times faster.

6.6 POSITIONING THE QUBO

So, on classical hardware, the direct MILP is the better tool. It finds the exact answer, proves that the answer is optimal, and does so in milliseconds. The QUBO encoding is still useful for the reason it was built in the first place. Quantum and quantum-inspired annealers take their input in this form, and until we can test such hardware, the projected annealer acts as a good classical substitute that solves the encoded problem almost exactly. We say this openly in the report, and the interactive engine of Chapter 12 shows the same note beside each comparison so that the formulation is not oversold.

Chapter 7

Trust-Aware GNN Inference and Task Assignment

7.1 ROLE OF THE INFERENCE STAGE

The pruner chooses the links the fleet will use, and the inference stage chooses which rovers will do the work. Because it runs on the fog device in every cycle, it has to be cheap. It looks at two kinds of information: what a rover is like on its own (trust, battery, free compute and capability), and where it sits in the kept graph, which tells us how reliable its neighbours are and how easily its results can get to the root.

7.2 NODE FEATURES AND MESSAGE WEIGHTS

Each rover i starts from the feature vector

$$h_i^{(0)} = [T_i,\ B_i,\ 1 - \ell_i,\ q_i] \in [0,1]^4, \tag{7.1}$$

trust, battery, available compute and capability. Messages along a kept edge (i,j) are weighted by

$$a_{ij} = T_{ij}\, L_{ij}, \tag{7.2}$$

the product of the edge trust (the smaller endpoint trust) and the measured link quality. As a result, a neighbour connected through a trusted, high-quality link has more influence on a rover's aggregate than a neighbour behind a poor link. Pruned edges have no influence at all, and this is how the pruner's choice ends up affecting the scores.

7.3 TWO-LAYER MESSAGE PASSING

Each layer forms the trust-weighted mean of the neighbours' states and combines it with the rover's own state:

$$m_i = \frac{\sum_j a_{ij}\, h_j}{\sum_j a_{ij}}, \qquad h_i' = \mathrm{ReLU}(W_s h_i + W_n m_i + b), \tag{7.3}$$

with $m_i = 0$ for a rover without kept neighbours. After two layers a logistic readout produces the raw score

$$s_i = \sigma(w^\top h_i^{(2)} + \beta). \tag{7.4}$$

The default weights are deliberately simple: $W_s = 0.8\,I_4$, $W_n = 0.2\,I_4$, $b = 0$, two layers, and a readout $w = (0.60, 0.15, 0.10, 0.15)$ over trust, battery, compute and capability with $\beta = 0$, dominated by trust.

The self weight dominates for a reason found empirically: with equal weighting, neighbour information washed out the per-rover differences that assignment depends on, and every rover in a cluster scored alike.

7.4 GATES, CRITICALITY AND DELIVERY AWARENESS

The raw score still has to be adjusted before it can be used for a decision, and we apply three adjustments.

Resource gates. A rover is ineligible for a task, with score zero, if its trust is below the task's minimum (0.25 in the simulator), its battery is below 15% or its free compute is below the task's need (0.3). Gates are hard because these failures are not matters of degree: a rover with an empty battery cannot execute, however good its neighbourhood.

Criticality. For mission-critical tasks (criticality above 0.5) the score is multiplied by $(0.5 + 0.5\,T_i)$. The scaling acts on the probability rather than on the logit, so its direction does not depend on the sign of the logit, and low-trust rovers are penalised more strongly exactly when failure is most costly.

Delivery awareness. The final score is multiplied by $(0.5 + 0.5\,p_i)$, where p_i is the delivery factor of Chapter 3: the weakest link on the widest path from i to the root in the kept graph. A rover that cannot reach the root at all ($p_i = 0$) is ineligible, because its result could never be delivered. The multiplier is only partial so that the root, whose delivery factor is always one, does not absorb every task merely by being the root.

Deadlock fallback. If the trust gate would leave a task without any eligible rover, for example early in a mission when every estimate is still near the prior, the trust gate is relaxed for that task while the resource gates are kept. The mission therefore never stalls on its own caution.

7.5 CAPACITY-AWARE GREEDY ASSIGNMENT

Once every rover has a score for every task, the tasks are assigned greedily, and no rover or task is used twice. We sort all (score, task, rover) triples with a positive score from highest to lowest, breaking ties by task name so that the result is repeatable. A triple is accepted when its task has not been assigned yet and its rover still has spare capacity, which by default means one task per rover per cycle. Greedy matching is not optimal

in every case. With one task per rover and only a few tasks per cycle, though, it comes close, and it runs in microseconds.

Every assignment increases the rover's load, and the load then falls again over the next few cycles. Before we added this, the most trusted rover was given every task until it ran out of compute.

Exploration. With probability $\varepsilon = 0.15$ per task, the assigned rover is replaced by a random eligible one. Exploration keeps trust estimates honest, because a rover that is never chosen is never observed, and the explored assignments are tagged: they form the random-policy, selection-bias-free subset used to train the readout in Section 7.8.

7.6 THE ONLINE TRUST MODEL

Trust is learned from outcomes. After each cycle, every assigned rover is updated with an exponentially weighted moving average,

$$T_i \leftarrow \alpha\, T_i + (1 - \alpha)\, \mathrm{obs}_i, \tag{7.5}$$

where obs_i is 1 if the rover executed its task and 0 otherwise (an optional link-evidence term can be mixed in with weights 0.7 and 0.3). Rovers that were not observed age slowly towards the neutral prior,

$$T_i \leftarrow T_i + \delta\, (0.5 - T_i), \tag{7.6}$$

with $\alpha = 0.5$ and $\delta = 0.1$ in the simulator.

Example 7.1 (Rehabilitation after one failure)**.** A rover at trust 0.5 fails once: Eq. (7.5) drops it to 0.25, which sits on the trust gate. If it is then idle, aging restores it as $0.5 - 0.25 \times 0.9^k$ after k cycles, reaching 0.35 after five cycles and 0.41 after ten. If there were no aging, the rover would stay at 0.25, it would never be picked, and so we would never see it again. One unlucky failure would shut it out for good, and this did happen in the simulator before aging was added.

We made the target of the trust model hard on purpose. In the simulator the hidden reliability is spatially correlated, because rovers that are close to each other share terrain and interference, and it can also drift over time. Trust therefore has to follow a target that moves and has structure, which is harder than estimating the bias of a fixed coin. The spatial correlation is also what gives message passing something to exploit: neighbours' outcomes carry information about a rover that has not been observed recently.

7.7 EDGE LIFECYCLE

A link that has been pruned is still remembered. The candidate graph is rebuilt from telemetry every cycle, so a pruned link can always return, and a small state machine tracks each link's recent history: a link is *active* while kept, *suspect* after one prune, in *probation* after two and *removed* after four consecutive prunes, and returns to *active* as soon as it is kept again.

Links in *probation* receive a freshness floor of 0.4. Without it, a link that was pruned for a few cycles would also go unobserved, its freshness would decay, its utility would fall further, and it would sink to the bottom of the ranking for good. The floor gives such links a fair re-evaluation when conditions improve.

7.8 THE CLOUD-TRAINED READOUT

The readout (w, β) of Eq. (7.4) can be trained in the cloud by logistic regression on pairs of hidden states and observed outcomes, which would turn the hand-set scorer into a learned model while keeping fog-side inference unchanged. Training data is subject to selection bias, because outcomes are observed only for rovers the policy chose, so training uses only the exploration-tagged samples, which were assigned at random. With that unbiased but small dataset (78 samples), the trained readout did *not* improve held-out mission success over the fixed weights (32.8% against 36.7%, a difference of -3.9 points with 95% CI $[-11.7, +11.7]$ and $p = 0.49$). We report this as a null result and did not keep tuning until the numbers looked better. Collecting more exploration data is part of the future work.

7.9 ABLATION ARMS

To see how much each part helps, we can swap out the scorer and keep the gates, delivery awareness and assignment exactly as they are. There are three replacements. In *trust-only* the score is simply the trust value, which is the bandit-style baseline [4]. *No-graph* uses the same readout on the raw features with no message passing, and *random* picks rovers at random. Chapter 10 gives the paired comparison of these arms, and the 3D engine can run all three next to a live mission.

7.10 COMPUTATIONAL COST

With n rovers and $|E'|$ kept edges, one layer needs $O(n + |E'|)$ small vector operations. On our evaluation machine the whole stage took about 0.2 ms for ten rovers and about

1 ms for fifty. This is far inside the time limit that shaped the architecture. Inference is not what slows the pipeline down; as Chapter 10 shows, pruning is.

Chapter 8

Closed-Loop Simulation and Evaluation Methodology

8.1 PURPOSE AND DESIGN PRINCIPLES

All quantitative end-to-end claims come from a closed-loop statistical simulator, a synthetic dynamic-graph world inspired by rover networking rather than a wireless channel model. Three principles governed it: *causality* (delivery runs over the kept graph and execution depends on hidden reliability, so both pruning and assignment affect success); *hidden truth* (the simulator knows each rover's reliability but the pipeline never sees it, so learned trust can be scored against the truth); and *paired comparisons* (every arm runs on identical scenarios from the same seed).

8.2 THE WORLD MODEL

Rovers and mobility. Rovers start at uniformly random positions in a 10×10 area and take a Gaussian random-walk step (standard deviation 0.8) every cycle, clipped to the area. Each rover has a battery that starts between 50 and 100%, drains by 1 to 5% per cycle and recharges by 3% when lightly loaded; a load that rises by 0.15 with each assignment and halves every cycle; and a static capability between 0.4 and 1.0.

Hidden reliability. Each rover's reliability is drawn from a base level between 0.35 and 0.75 plus a spatially correlated field made of three Gaussian bumps of random sign and amplitude up to 0.3 (width 2.5), clipped to $[0.05, 0.98]$. Nearby rovers therefore share good or bad conditions, which is what makes neighbourhood information useful to a GNN. Optionally, reliability drifts as a random walk with a chosen standard deviation per cycle, so trust must track a moving target.

Tasks and mission roster. A fixed number of tasks arrives each cycle (two by default), each with a criticality drawn between 0.2 and 0.9. A fixed mission-critical roster, the root plus a few randomly chosen rovers, is drawn once per scenario. The roster is deliberately not redefined every cycle as "whoever happens to be reachable", which would make the connectivity guarantee trivially satisfied.

8.3 THE RADIO MODEL

Two rovers can link when they are within the communication range. The chance that a link is heard in a given cycle is $1 - \pi$, where π is the packet loss, set to 5% unless stated otherwise. An observed link refreshes its quality, $L_e = \text{clip}(1 - d_{ij}/d_{\max} + \mathcal{N}(0, 0.05^2))$, and its timestamp; an unobserved link keeps its old quality and grows stale, with freshness $A_e = \exp(-\Delta t_e/3)$ in cycles. The edge cost follows Eq. (3.1). So packet loss does not remove links from the graph. What it does is make the pipeline's information about them older and less trustworthy, which is also what tends to go wrong with real radios.

8.4 FAULTS

We test the trust model and the pruner with scripted faults. A *reliability drop* lowers the hidden reliability of one rover, or of all of them. An *intermittent* fault switches that drop on and off every other cycle. A *packet-loss spike* makes the link information of the whole fleet go stale. A *battery drain* empties one rover until it can no longer carry out tasks, and a *position jump* suddenly moves a rover to a new place. In the default scenario the most reliable rover is degraded for four cycles, and a packet-loss spike starts one cycle after that. This is the hardest case for a system that has learned to depend on that rover.

8.5 EXPERIMENTAL ARMS

8.5.1 PRUNER ARMS

Six pruners share the same utility, Eq. (3.2), so they differ only in how they choose:

- **full graph:** keep every candidate link;
- **threshold:** keep links with $U_e \geq 0.4$;
- **top-K:** keep the best half of the links by utility;
- **threshold+repair** and **top-K+repair:** the same greedy choice followed by the same feasibility repair the QUBO path uses, enforcing degree caps, the budget and connectivity of the mission roster; if the budget is infeasible, the repair falls back to a minimum-cost spanning tree and reports the residual violation rather than returning an empty graph;
- **QUBO:** the projected annealer with the relaxation ladder of Section 5.7.

The repaired arms are essential for fairness. A table that compares the constrained QUBO only against constraint-blind pruners would credit the QUBO with guarantees its opponents never tried to provide.

8.5.2 BUDGET AND DEGREE SETTINGS

The budget is either a fixed number of units K or a fraction β_K of what the full graph costs in the current cycle. The second option keeps the pressure similar from one cycle to the next even when the graphs are different. Unless an experiment changes it, the degree cap is three. Section 10.2.1 explains why the second budget mode became necessary.

8.5.3 CLOUD AMORTISATION

The QUBO can be solved every k cycles instead of every cycle. Between solves the last decision is reused, intersected with the live graph so that no vanished link is ever kept. This models a cloud solver feeding a fog device on a slower cadence.

8.5.4 ASSIGNMENT ARMS

The four assignment arms of Chapter 7, GNN, no-graph, trust-only and random, share gates, delivery awareness and capacity, and differ only in the scorer.

8.6 METRICS

Table 8.1 defines every metric reported in Chapter 10. Four of them, offload, delivery failure and the precision and recall of delivery-critical pruning, are usually missing from pruning studies and were added specifically to test what the guarantees buy.

Table 8.1: Metrics computed by the simulator.

Metric	Definition
Mission success	fraction of tasks both executed and delivered
Offload rate	fraction of task slots actually assigned on the kept graph
Delivery-failure rate	failed deliveries divided by attempted deliveries
Pruning precision / recall	agreement of kept edges with *delivery-critical* edges (below)
Trust Spearman	rank correlation between learned trust and hidden reliability
Trust AUC	how well trust separates rovers above and below median reliability
Violations	per-run counts of excess degree, budget overruns and disconnected mission rovers
Stage latency	median of repeated timings of build, solve, GNN and assignment
Recovery time	cycles until success returns to its pre-fault level
Relaxation usage	cycles spent on each rung of the relaxation ladder

Definition 8.1 (Delivery-critical edge)**.** In a given cycle, a candidate edge is *delivery-critical* if removing it from the full candidate graph strictly reduces the widest-path delivery factor p_i of at least one mission-critical rover.

The labels are recomputed every cycle on the live candidate graph, so they are a counterfactual ground truth that does not depend on any pruner. Recall then asks how

many of the links that matter for delivery a pruner kept, and precision how many of its kept links were ones that matter.

8.7 STATISTICAL METHODOLOGY

Results are averaged over seeds, and differences between arms are always computed *per seed*, because two arms run on the same scenario share most of their randomness. For per-seed differences $d_1, \ldots, d_S$ the report gives three numbers.

1. The mean difference $\bar{d}$.
2. A paired bootstrap 95% confidence interval: resample the S differences with replacement 10 000 times and take the 2.5th and 97.5th percentiles of the resampled means [7].
3. A sign-flip permutation p-value: under the null hypothesis of no difference, each d_s is equally likely to have either sign, so the p-value is the fraction of 20 000 random sign assignments whose mean is at least as extreme as $\bar{d}$.

A difference is called significant when the interval excludes zero and $p < 0.05$. With 8 to 16 seeds per arm the power is modest, and the report says so wherever a margin is not significant rather than drawing a trend through noise.

8.8 SCALING AND PRESSURE EXPERIMENTS

Two further experiment drivers run outside the closed loop.

Scaling. Range-limited sparse graphs, in which each rover links to its nearest neighbours as real short-range radios would, are built for fleets of 10 to 50 rovers. At each size the driver records the numbers of edges and variables, the memory used by the matrix, the time to build and to solve, the energy reached by the projected annealer and by a repaired greedy baseline, and the GNN's latency on both the full and the pruned graph. Connectivity is enforced only up to 20 rovers, because the dense flow encoding grows quadratically, and the report never draws one curve across that transition.

Constraint pressure. In the pressure sweep we keep the scenarios the same and tighten one constraint at a time. The budget fraction takes the values $\beta_K \in \{0.35, 0.5, 0.75, 1.0\}$, the degree cap $D \in \{2, 3, 4\}$ and the roster size $|R| \in \{3, 5, 8\}$. Every setting is run with six pruner arms, 16 seeds and 20 cycles per run. Results are saved after each run, so a sweep that gets interrupted can carry on from where it stopped.

We measured all timings on an AMD Ryzen 7 5800H (16 threads, 8 GB of memory) using Python 3.14 and NumPy with its reference linear algebra library. The conclusions rest on the *ratios* between times more than on the absolute values.

Chapter 9

Verification and Testing

9.1 PHILOSOPHY: A HOSTILE AUDIT

It is not hard to make a research pipeline look good, but it is much harder to make it trustworthy. We therefore set up verification as a *hostile* audit, with about five hundred automated checks that each try to break one of the claims in this report. We followed three rules.

1. **Independent answer keys.** Where we could, the expected value comes from code or mathematics that has nothing in common with the code being tested. Examples are a term-by-term re-derivation of the energy, an optimum worked out by hand, an exact solver, or enumeration of every case.
2. **Floors, not snapshots.** Quality checks set a minimum level of performance, such as "at least 90% exact", instead of expecting one exact number. A real regression then makes the suite fail, while small numerical noise does not.
3. **Negative results stay visible.** When a check documents a weakness, for example how often the samplers fail, we keep it and report it. It is not removed just because the result is inconvenient.

9.2 THE SHARED TOOLBOX

Five helper functions are used across the suite. The first generates random connected instances and can optionally add *tight* caps that sit close to the edge of feasibility. The second is an *independent energy* that works out every term of Eq. (4.1) directly, without using the matrix. The third is brute force over all bit strings, for very small instances. The fourth enumerates every subset of optional edges, computes the slacks in closed form and checks connectivity directly, and it provides the ground truth for small instances. The last is the normalised gap of Eq. (5.2).

9.3 THE FOURTEEN SECTIONS OF THE SUITE

The suite is summarised in Table 9.1. Each section tests one question. Taken together they cover the encoding, the solvers, the exact baselines, the GNN, the simulator and the regression cases.

Table 9.1: The fourteen sections of the test suite.

	Section	What is checked
A	Construction invariants	symmetric, finite Q; unique variable names; matrix energy equals the independent energy on random states
B	Brute force vs SA	two answer keys agree to 10^{-9}; brute-force optimum is feasible; bit-level SA within a gap floor; repair never fails
C	Analytic optima	with constraints removed, the optimum equals a closed-form sum of per-edge decisions, with and without stability
D	Scenario matrix	80 larger random worlds: raw SA may be infeasible, but after repair zero instances remain infeasible
E	Validation and edge cases	twelve kinds of bad input raise clear errors; empty problems, zero budget and default degree caps behave as designed; same seed gives identical output
F	Scaling sanity	feasibility and a time limit at 8, 12 and 16 rovers; matrix memory reported
G	Domain scenarios	seven hand-built stories with an unambiguous right answer
H	Solver audit	four samplers on the actual Q against a doubly verified ground truth; projected annealer must exceed a 90% floor
I	Trust-GNN layer	permutation equivariance, score ordering, hard gates, neighbourhood effect, assignment validity, trust dynamics, end-to-end gain over random, readout training
J	Optimality gap	100% exact on small instances; certified bounds and best-known gaps on large ones
K	Simulation harness	every pruner and fault runs; freshness decays under total packet loss; rank correlation and edge lifecycle behave correctly
L	Regressions	minimal reproductions of every bug found during review stay fixed
M	MILP ground truth	projected annealer against exact MILP optima; exact-rate floor of 80%
N	Sensitivity	exact rate under perturbed temperatures, cooling, restarts, penalties and move mix

9.4 SELECTED CHECKS IN DEPTH

9.4.1 ANALYTIC OPTIMA

Section C provides the strongest checks in the suite, because their expected answers come from a formula rather than from any code. If degree caps, the budget, connectivity and stability are all removed, every edge becomes an independent choice: keeping it changes the energy by $-U_e + \lambda$, so the optimum is $\sum_e \min(0, -U_e + \lambda)$. With stability restored, each edge chooses between dropping, at cost $\eta(x_e^{t-1})^2$, and keeping, at cost $-U_e + \lambda + \eta(1 - x_e^{t-1})^2$, so the optimum is the sum of the per-edge minima. Brute force over the full matrix must reproduce both sums to 10^{-9} on twenty random instances each. A failure here while section B passed would mean that the matrix encodes *something* consistently, but not the intended something.

9.4.2 DOMAIN STORIES

Section G contains seven small situations where anyone can check what the right outcome should be. If a required rover can only be reached through a very poor *bridge* link, that link has to stay, since dropping it costs a 500-point flow penalty. A weak path that is redundant has to be pruned. Out of two identical links, the one kept in the previous cycle has to stay (*hysteresis*). A hub with a cap of 2 has to drop its worst leaf. Ten identical links have to give the same answer every time. Under a tight budget, two cheap links should be chosen over one expensive link. Finally, a chain of very poor links that are all needed has to be kept in full. These cases are easy to explain, which makes them useful when someone reviews the work.

9.4.3 THE SOLVER AUDIT

Section H is set up so that every solver is judged by the same rules. Its 25 instances are small enough to have a known answer, and that answer is itself checked: on five instances the exact linearised solve has to agree with enumeration to within 10^{-7}. All samplers get the real matrix, the same warm start and the same scoring. One conversion step needs care. External samplers use the upper-triangular form $\sum_i Q_{ii}x_i + \sum_{i<j} Q_{ij}x_ix_j$, but our matrix is stored as a full symmetric matrix, so the off-diagonal entries are doubled before the matrix is passed on. If this were done wrongly, the samplers would be tested on a different problem. The section then records the 48 to 64% exact rates from Chapter 5. For the production annealer, which is not part of the scientific comparison, it only checks a minimum level.

9.4.4 CERTIFIED BOUNDS ON LARGE INSTANCES

On its larger instances, Section J has no known answer to compare against, so it uses a bound instead. Taking each edge on its own, the lower of its keep cost and its drop cost is the best that edge could ever contribute. Since penalties are non-negative, the sum of these per-edge minima is a certified lower bound $LB \leq H^*$. Because $H^* \leq H_{\mathrm{SA}}$, the quantity $(H_{\mathrm{SA}} - LB)/\max(1, |LB|)$ is an *upper bound on the true gap*. If it is small, the solver is provably close to optimal without any exact solve. This bound is reported next to the gap to the best-known solution.

9.4.5 REGRESSIONS

For every bug found in the external review rounds there is now a small test that reproduces it. One example is a chain graph with connectivity constraints and no warm start, which the annealer used to fail on. Greedy growth could not get past the infeasible "half a chain" states. Starting from random spanning trees fixed the problem, and a test keeps it from coming back.

9.5 TESTING THE DEPLOYMENTS

The two deployments carry their own tests, written in the same spirit.

3D engine motion tests. Seventeen property tests check the mathematics of Chapter 12: angle wrapping and short-arc interpolation; agreement of the pose integrator with the closed-form circular arc and its exactness for straight lines; that a right turn drives the left wheel faster; that each wheel's contact point travels exactly its rim distance along an arc; that wheel-speed saturation preserves curvature; that spawn layouts never overlap and are deterministic; that over 1 800 frames at triple speed and up to 16 rovers no two footprints ever overlap and no rover leaves the arena; that a dense swarm never deadlocks permanently; that corrupted frame times and planner data are rejected; and the widest-path, parameter-snapping, advisory and bootstrap utilities.

Planner service tests. Nineteen tests check the service that connects the 3D engine to the pipeline: parameter, position, fault and comparison validation; the shape and consistency of a planning step; the behaviour with no links in range, an impossible degree cap and a budget below the cheapest tree; live tuning without restarting a mission; session capacity and idle eviction; fault delivery to every paired arm; identical topology for the paired baselines; the solver comparison on the same matrix; and the HTTP round trip, including rejection of malformed requests.

Equivalence of instrumentation. The planner service needed a few read-only hooks in the simulator. Their addition was verified by running the simulator with and without them on five scenarios, including the repair pruners and the budget-fraction mode, and checking that every cycle's results are identical.

9.6 REPRODUCIBILITY PRACTICES

Every reported number can be regenerated provided the seeds, instance generators and package versions are recorded with each run, and the experiment drivers write their results incrementally so that long sweeps can be resumed. Comparisons between pruners and between assignment policies are always made on identical scenarios. Finally, the suite is run in full before any change is accepted, and optional solver backends that are not installed cause their checks to be skipped, never silently passed.

Chapter 10

Results and Discussion

10.1 OVERVIEW

This chapter reports the end-to-end results: pruning, constraint pressure, assignment, amortisation, scaling, and trust and fault behaviour; the solver results were given in Chapters 5 and 6. Negative results are reported with the same prominence as positive ones, because they define where the architecture should and should not be used. Table 10.1 gives the whole picture.

Table 10.1: Scoreboard of the main quantitative results.

Measurement	Result
Encoding agreement with independent re-derivation	within 2.3×10^{-10}
Exact-solve rate of four annealing samplers	48 to 64%
Projected annealer: small instances / against MILP	100% / 95% exact
MILP against projected annealer, time	about 100 times faster
Trust-aware GNN against trust-only	+8.1 pts, CI [+4.2, +11.7], $p = 0.016$
Trust-aware GNN against random / no-graph	+13.8 pts / +1.7 pts (n.s.)
QUBO pruner against best repaired baseline	−0.1 to −4.1 pts, never ahead
QUBO degree violations at any pressure	zero
Cloud-trained readout against fixed weights	−3.9 pts (n.s.)
Recovery after an injected fault	3 to 4 cycles

10.2 THE PRUNER COMPARISON

Table 10.2 compares the six pruners on identical scenarios of 10 rovers, 20 cycles and 16 seeds with drifting reliability, at the project's original fixed budget of $K = 15$ units.

Table 10.2: Pruner comparison, 10 rovers, 20 cycles, 16 seeds, fixed budget $K = 15$. Violations are mean counts per run (degree / budget / connectivity). †See Section 10.2.1.

Pruner	Success	Offload	Del. fail	Solve	Spearman	AUC	Violations
full graph	36.0%	1.000	0.385	none	0.271	0.683	158.8 / 20.0 / 0.4
threshold	37.0%	0.955	0.340	< 0.01 ms	0.254	0.624	131.7 / 20.0 / 2.3
top-K	36.4%	0.898	0.309	< 0.01 ms	0.251	0.683	53.6 / 20.0 / 10.0
threshold+repair	34.3%	1.000	0.396	0.4 ms	0.249	0.640	4.7 / 20.0 / 0.4
top-K+repair	34.8%	1.000	0.392	0.2 ms	0.242	0.638	3.4 / 19.9 / 0.4
QUBO (ours)	32.2%	0.998	0.418	1435 ms	0.272	0.669	0.0 / 19.9† / 0.4

10.2.1 THE AUDIT CAUGHT THE AUDIT

The relaxation bookkeeping of Section 5.7 exposed a problem in the project's own earlier results. At $K = 15$ the production pruner used the *no-budget* rung in 319 of 320 cycles: at ten rovers a budget of 15 units is below the cost of any connected subgraph, so the ladder dropped it, and the QUBO row of every earlier pruner table was in effect unconstrained. This is precisely what the bookkeeping was built to catch, and an argument for always logging relaxations rather than asserting feasibility. All constraint-relevant conclusions were re-run at a budget that is both binding and feasible, $K = 0.75\times$ the full-graph cost, where the QUBO solved with every constraint in 320 of 320 cycles (Fig. 10.1).

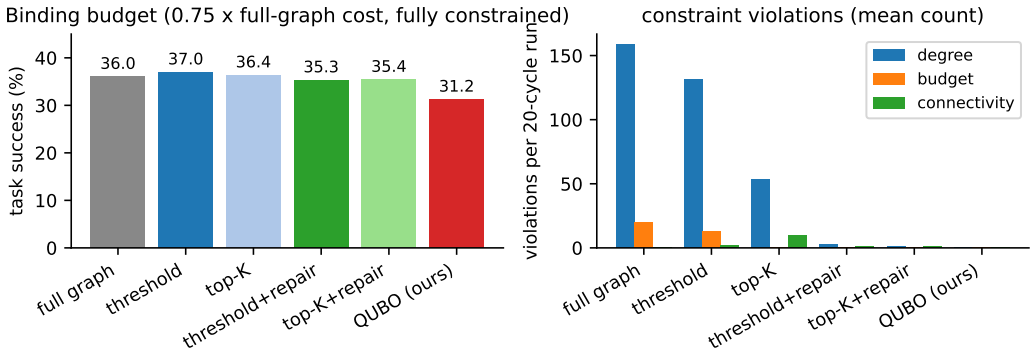


Figure 10.1: Mission success and constraint violations of the six pruners at a binding, feasible budget (0.75× the full-graph cost).

10.2.2 WHAT THE GUARANTEES COST AND BUY

At the binding budget the QUBO pruner is behind on raw success by 4.1 points against threshold+repair (95% CI $[-6.4, -1.9]$, $p = 0.004$), by 4.2 points against top-K+repair (CI $[-7.0, -1.5]$, $p = 0.017$) and by 5.7 points against the constraint-blind threshold pruner ($p = 0.002$). Comparing only against constraint-blind pruners had overstated the cost of the guarantees by about two points, but the corrected gap remains significant. What the QUBO buys is the guarantee itself: it is the only arm with zero violations (the repaired baselines leave 1.1 to 3.1 degree and 0.8 to 0.9 connectivity violations per run), and it has the best trust Spearman (0.295 against 0.251 to 0.271), though not the best AUC. The delivery metrics show its weakness (Fig. 10.2): a delivery-failure rate 4.4 points worse than threshold+repair (CI $[+1.4, +8.0]$) and 11.8 points worse than top-K, and a recall of delivery-critical links of 0.67, against 0.82 for the repaired baselines and 0.93 for the threshold pruner; only the constraint-blind top-K, which keeps half of the links, is lower (0.56). The edge-local utility ranks links by their own quality rather than by how much delivery depends on them, so the optimum sometimes prunes exactly the redundancy that bottleneck paths need.

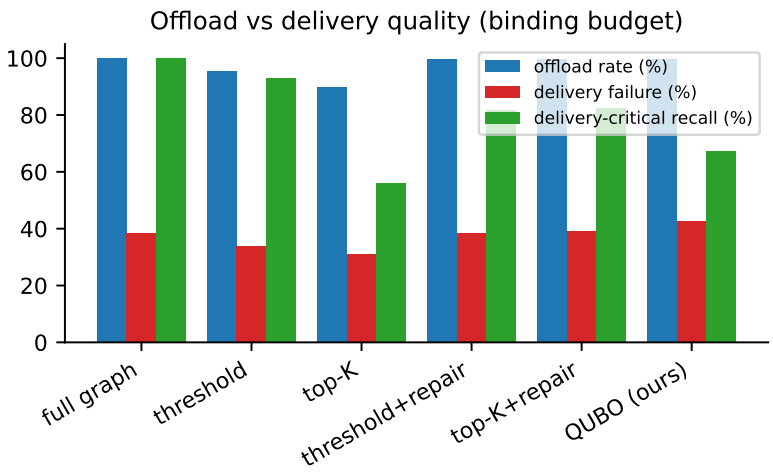


Figure 10.2: Offload rate, delivery-failure rate and delivery-critical recall of each pruner at the binding budget.

10.3 CONSTRAINT PRESSURE

Does the guarantee ever turn into a success advantage when constraints bite hard? Table 10.3 and Fig. 10.3 answer it.

Table 10.3: Constraint-pressure sweep, 10 rovers, 20 cycles, 16 seeds per cell; differences in percentage points with permutation p-values.

Constraint	Level	QUBO	Best repaired	QUBO − best
budget β_K	0.35	34.6%	34.7%	−0.1 (p = 1.00)
	0.50	31.6%	35.2%	−3.7 (p = 0.033)
	0.75	31.3%	35.4%	−4.1 (p = 0.004)
	1.00	31.7%	35.3%	−3.7 (p = 0.020)
degree D	2	30.9%	34.3%	−3.3 (p = 0.15)
	3	32.2%	34.3%	−2.1 (p = 0.10)
	4	33.3%	34.8%	−0.9 (p = 0.52)
roster $\|R\|$	3	32.2%	34.3%	−2.1 (p = 0.10)
	5	32.4%	35.1%	−2.7 (p = 0.058)
	8	34.6%	37.8%	−3.2 (p = 0.020)

The answer is not the one the guarantee story predicts. At extreme budget pressure ($\beta_K = 0.35$) the QUBO ties the best repaired baseline, where even the repaired arms cannot fully meet the budget. At every looser setting it is behind by 2 to 4 points, and in the degree and roster sweeps the gap narrows only as the constraint relaxes. There is no crossover, even where the constraint-blind threshold pruner breaks the budget in 20 of 20 cycles and, at $D = 2$, exceeds degree caps 165 times per run. What holds at every level is the guarantee: zero degree violations, budget residuals only where the budget is infeasible (and then reported), and connectivity misses only where no pruner could connect the roster. The guarantees are worth their cost when violating a constraint is disqualifying (regulatory, safety or contractual), not when success is the only currency.

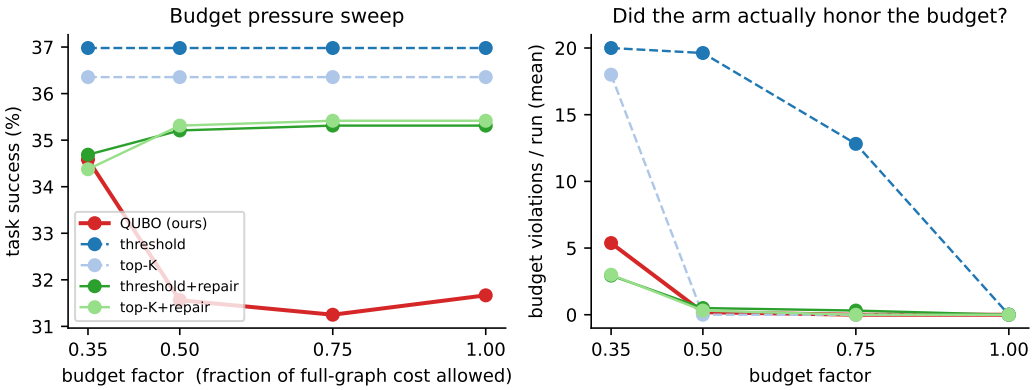


Figure 10.3: Budget pressure sweep: success by arm (left) and whether each arm honoured the budget (right).

10.4 ASSIGNMENT ABLATION

Table 10.4 isolates the assignment stage on identical scenarios. The trust-aware GNN beats trust-only by 8.1 points with an interval that excludes zero and a significant permutation test, and beats random by 13.8 points. Its margin over the no-graph readout is positive but not significant: at ten-rover density most of the value lies in the trust model, the gates and delivery awareness, and message passing adds a small extra that this sample cannot separate from noise.

Table 10.4: Ablation of the assignment stage with 10 rovers, 20 cycles and 8 seeds. Differences are computed per seed, in pairs.

Policy	Success	Difference to the GNN
random	26.0%	+13.8 pts
trust-only	31.7%	+8.1 pts, CI [+4.2, +11.7], $p = 0.016$
no-graph readout	38.1%	+1.7 pts, CI [−2.1, +6.9], $p = 0.736$
trust-aware GNN (ours)	39.8%	

10.5 AMORTISATION AND SCALING

A solve of more than a second cannot run synchronously on the fog device, so the cloud can solve every k cycles while the fog device reuses the last decision, intersected with the live graph (Table 10.5). Using a decision that is two cycles old costs less than one point of success and cuts the number of solves in half. For $k \geq 4$ the differences between seeds are larger than the effect we are trying to measure, so we report the numbers as they are instead of drawing a smooth curve through them.

Figure 10.4 puts the time spent on pruning next to the time spent on inference. Solving takes about five orders of magnitude longer. At these small fleet sizes, then, the reason to prune is the guarantees it gives and not any saving in time. It also means

Table 10.5: Solving in the cloud every k cycles, with 10 rovers, 20 cycles and 6 seeds.

Solve interval k	Success	Solves per cycle	Amortised solve (ms)
1	30.3%	1.00	1279
2	29.4%	0.50	618
4	26.9%	0.25	273
8	31.9%	0.15	151

the solve should run in the cloud or only every few cycles, which is what the three-tier design planned for.

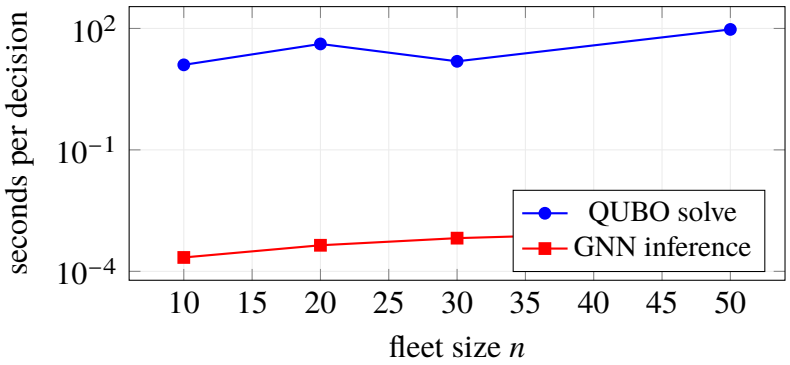


Figure 10.4: Wall-clock time for each decision on the scaling instances. The time drops at $n = 30$ because connectivity is no longer enforced above 20 rovers.

10.6 TRUST QUALITY, TRAINING AND FAULTS

The learned trust values follow the hidden reliability with a Spearman correlation between 0.24 and 0.30 and an AUC between 0.62 and 0.68. Comparing trust quality across pruners is not fair, because each pruner produces its own set of outcomes, and we point this out instead of leaving it unsaid. As reported in Section 7.8, the readout trained in the cloud did not do better than the fixed weights. During the scripted fault, the trust of the degraded rover goes down, tasks are moved to other rovers, and success gets back to where it was before the fault 3 to 4 cycles after the fault ends.

10.7 DISCUSSION AND LIMITATIONS

Three findings cut against the naive version of the project's own pitch and are its most useful results: penalty-encoded constrained QUBOs are easy to write and hard to sample; on classical hardware the QUBO route is dominated by a direct MILP; and at small scale the constrained pruner trades 2 to 4 points of success for its guarantees. The limitations are equally concrete: 8 to 16 seeds per arm; a message-passing margin that is not significant; selection-confounded trust comparisons; a synthetic simulator with no channel model, hardware in the loop or quantum processor; a dense matrix that limits

the fleet to tens of rovers; and hand-chosen weights and penalties, tested for sensitivity but not tuned by a principled procedure.

Chapter 11

Physics Simulation in Gazebo with ROS 2

11.1 MOTIVATION AND SOFTWARE STACK

The statistical simulator is fast, seeded and exactly reproducible, but it is not a physics engine and it does not exercise the questions every deployment must answer: how the pipeline receives live state from a robot simulator, how it is packaged so that anyone can reproduce it with one command, and how its decisions can be observed while a world runs. To answer them the pipeline was implemented in Gazebo: four differential-drive rovers are simulated by Gazebo Harmonic, their odometry is bridged into ROS 2 Jazzy, and the full pipeline runs inside a ROS 2 node against that live stream, publishing its decisions once per second. The chapter reports an integration, not a new experimental result.

The integration has four layers (Fig. 11.1). Gazebo Harmonic simulates rigid-body physics with the ODE engine, the rover models and their differential-drive controllers, all described in SDF 1.9. A Gazebo–ROS bridge forwards the four odometry topics into ROS 2 *Odometry* messages, in the Gazebo-to-ROS direction only. ROS 2 carries them to the planner node, a thin adapter around the unchanged pipeline: the pipeline is not rewritten as ROS packages, because it is middleware-agnostic by design.

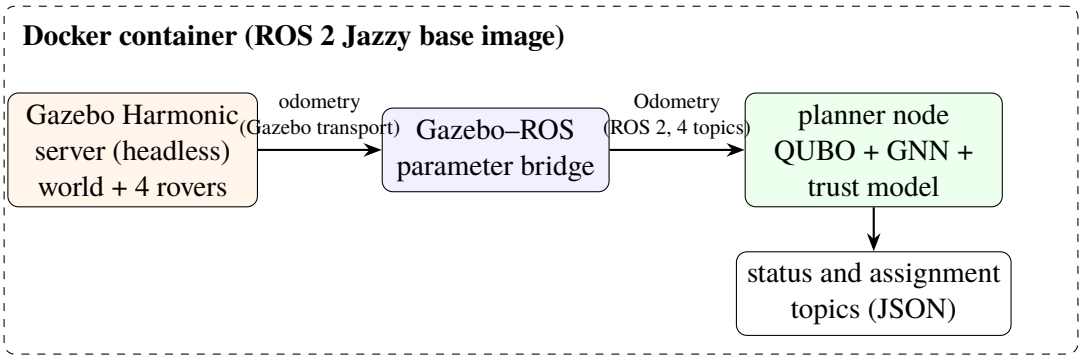


Figure 11.1: The Gazebo integration inside one container.

11.2 THE WORLD AND THE ROVER

The world is deliberately simple, so that every effect can be attributed (Table 11.1). Four instances of one rover model stand on a line, the same line-up as the planned physical testbed. Each rover is a 0.9 × 0.65 × 0.25 m, 3 kg body with two wheels of radius $r_w = 0.16$ m on revolute joints, a track of $L_w = 0.76$ m, and a differential-drive plugin

that turns velocity commands into wheel motion and publishes odometry at 20 Hz. The 3D engine of Chapter 12 uses the same dimensions, so the two simulators share one rover.

Table 11.1: Settings of the Gazebo world and of the rover model.

Element	Setting
Physics	ODE, step 1 ms, real-time factor 1.0, update rate 1000 Hz
Scene and ground	directional sun with shadows; static 30×30 m plane
Spawn poses (x, y, ψ)	R0 $(-3.0, 0.0, 0.00)$, R1 $(-1.0, 0.4, 0.25)$, R2 $(1.0, 0.1, -0.20)$, R3 $(3.0, 0.5, 0.10)$
Body	box $0.9 \times 0.65 \times 0.25$ m, 3.0 kg, inertia $(0.2, 0.2, 0.3)$ kg m^2
Wheels and joints	radius 0.16 m, width 0.08 m, 0.25 kg; revolute, unlimited
Drive plugin	differential drive, separation 0.76 m, odometry at 20 Hz

A differential-drive rover is commanded by a forward speed v and yaw rate ω; the plugin converts them into wheel rim speeds and back again for odometry,

$$v_L = v - \frac{\omega L_w}{2}, \quad v_R = v + \frac{\omega L_w}{2}; \qquad v = \frac{v_L + v_R}{2}, \quad \omega = \frac{v_R - v_L}{L_w}, \tag{11.1}$$

in Gazebo's convention of counter-clockwise positive yaw with the left wheel at positive y; each wheel turns at $v_{L,R}/r_w$. Chapter 12 uses the same relations in its own mirrored convention and integrates them exactly.

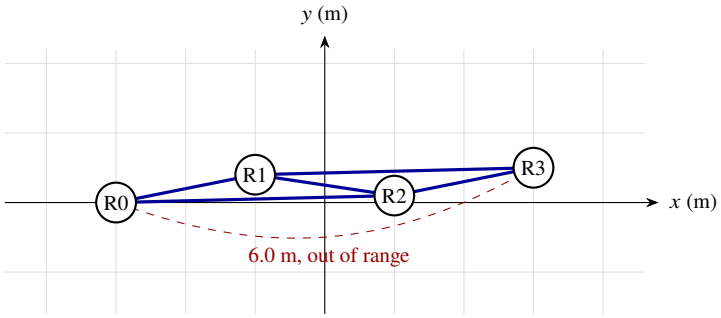


Figure 11.2: Top view of the Gazebo world: with a 5 m radio range the four rovers form five candidate links; the end rovers are 6 m apart and cannot link.

11.3 THE PLANNER NODE

The node subscribes to the four odometry topics, keeps each rover's latest planar position, and runs one pipeline cycle per one-second timer tick (Algorithm 1). Every cycle it publishes two JSON messages: a status message (cycle, number of rovers with odometry, candidate and kept edges, pruner mode, solve time, assignment and every rover's trust) and an assignment message.

Algorithm 1 One cycle of the Gazebo planner node

Input: latest odometry positions, trust model, previous kept edges

for each pair of rovers within the 5 m radio range **do**
    $P \leftarrow 1 - d/d_{\max}$; link quality $0.85P + 0.15$; cost $0.10 + 0.50(1 - P)$
end for
build the QUBO: all four rovers required, root 0, degree cap 3, budget 25 units
solve by projected annealing, warm-started from the previous kept edges
if no feasible solution **then** keep every candidate link and log a warning
end if
score rovers for two tasks (criticality 0.7 and 0.4) on the kept graph
multiply scores by $(0.5 + 0.5\,p_i)$ and assign greedily, one task per rover
sample execution and delivery; update trust; age, decay loads, drain batteries
publish the status and assignment messages

Four design choices are worth explaining. First, link quality depends only on distance, in a *deterministic* way. Gazebo simulates geometry but not radio signals, so this keeps repeated runs comparable while Gazebo still decides where the rovers are. Second, we use a short annealing schedule (temperatures 0.5 to 0.05, cooling 0.8, two restarts), which solves the four-rover problem in about 13 ms, much less than the length of a cycle. Third, the node begins from the spawn poses of the world, so it can make reasonable decisions while the bridge is still starting up. Its status message also says how many rovers have sent odometry, which makes a missing bridge obvious straight away. Fourth, if the physical graph ever becomes infeasible, the node keeps all the links and flags that cycle. The demonstration therefore never stops without telling the user why.

11.4 PACKAGING AND LAUNCH

Everything runs in one Docker container built on the official ROS 2 Jazzy image, with the Gazebo simulator, the bridge, the message packages and NumPy added and the project copied in, so the pipeline is importable as ordinary Python. A compose file defines one service, with the number of cycles passed as an environment variable. The launch proceeds as in Fig. 11.3: before building, it refuses to continue with less than 8 GB of free disk, because a truncated build of the large base image fails far from the real cause; after starting the simulator headless, it waits three seconds and, if the server has died, prints its log and exits. An optional flag starts the graphical client when a display is available.

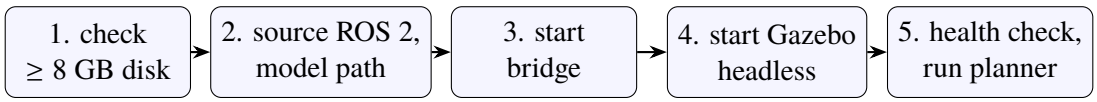


Figure 11.3: Launch sequence of the container.

11.5 BEHAVIOUR AND LIMITATIONS

Table 11.2 shows the node's decisions on the spawn line-up, obtained by running its cycle code offline with those poses, which are what the live node receives while the rovers remain where they spawned.

Table 11.2: Planner cycles with the rovers at their spawn positions. All 5 candidate links were kept in every cycle and each solve took 12 to 16 ms. The trust values are the ones after each cycle.

Cycle	Critical task	Routine task	T_0	T_1	T_2	T_3
0–2	rover 3	rover 0	0.73→0.92	0.50	0.50	0.73→0.92
3	rover 3 (fails)	rover 0	0.95	0.50	0.50	0.50
4	rover 1	rover 0	0.97	0.73	0.50	0.50
5–11	rover 1	rover 0	0.99→1.00	0.85→1.00	0.50	0.50

Two things stand out in this run. The first is that the pruner keeps all five links in every cycle. The line-up is short and every link is healthy, so each link's utility is far above the sparsity penalty, and neither the caps nor the budget are tight. Keeping every link is then the optimal choice. Pruning only starts to make a difference once weak or expensive links show up, as they do in the larger fleets of Chapter 10. The second is that trust clearly changes the assignment. Rover 3 gets the critical task while it is trusted, then fails in cycle 3, its trust falls from 0.92 to 0.50, and from the next cycle onwards the task goes to rover 1.

This integration also has clear limits. The node reads odometry but does not send any velocity commands, so for now the rovers stay at their spawn poses. The next step is a waypoint follower built on the controller from Chapter 12, which would make the graph change between cycles. Link quality has no radio model, outcomes are sampled from assumed reliabilities, and the world is written for four rovers. Even so, the unchanged pipeline runs against live simulator state inside standard robot middleware, is reproducible with one command, and exposes every decision on an inspectable stream.

Chapter 12

An Interactive 3D Simulation Engine

12.1 GOALS AND ARCHITECTURE

The statistical simulator answers questions with tables and the Gazebo world answers integration questions, but neither lets a person *watch* the pipeline think. The project therefore built its own interactive 3D simulation engine, which runs in a web browser (Fig. 12.1). We set ourselves a list of requirements for it. Every decision had to come from the real pipeline. The rovers had to move with correct differential-drive kinematics, smoothly, and they could never collide. Links had to follow the rovers in every frame and show which links are kept, which are only candidates, which have stretched out of range, and where data is flowing. All parameters had to be adjustable while the simulation runs, and the pipeline had to be compared fairly with baselines that it beats. Finally, the scene had to stay fast for any fleet size and offer orbit controls along with a game-style map showing every rover.

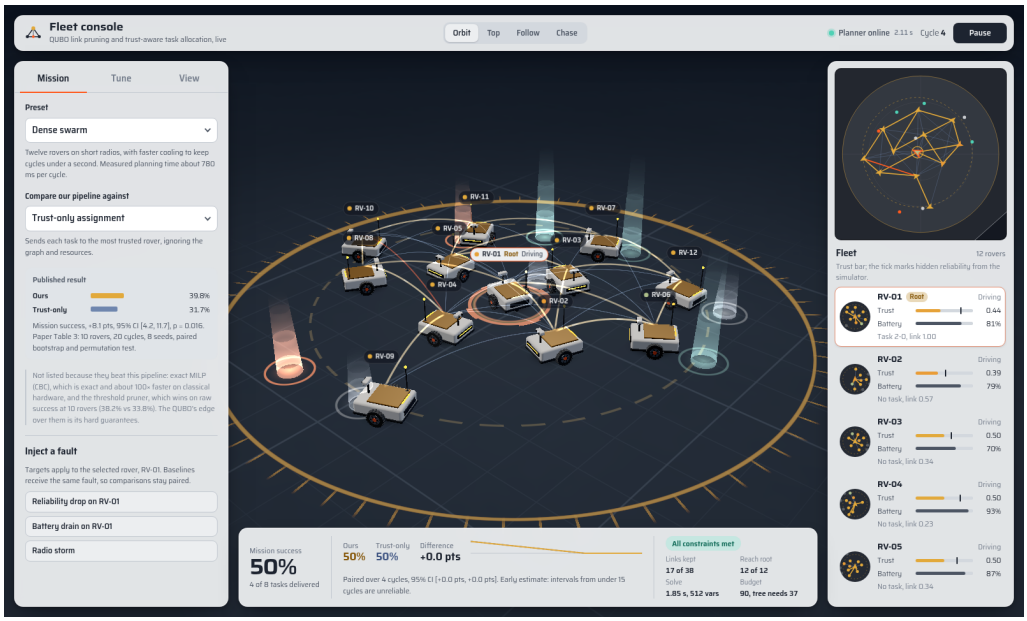


Figure 12.1: The engine running the twelve-rover *Dense swarm* preset. The mission controls are on the left and the 3D scene, with kept links and task beacons, is in the centre. The overview map and the radar for each rover are on the right, and the live comparison with trust-only assignment is along the bottom.

The work is divided between two parts that talk through one small interface. The browser is in charge of time, motion and rendering. All decisions are made by a planner service, written in Python, which wraps the closed-loop simulator without changing it.

Once per planning cycle the browser sends the rovers' live positions and the current parameters, and the service runs one pipeline cycle in which those positions replace the simulator's random walk, returning kept links, trust, assignments, task outcomes and diagnostics. Parameter limits, presets and the comparison catalogue live in one configuration document read by both sides, so they can never disagree about what is valid. Each parameter carries a *scope*: fleet size and seed restart the mission, planner parameters apply from the next cycle without losing learned trust, and speed, time scale and cycle interval apply instantly (Table 12.1).

Table 12.1: Tunable parameters, their ranges and scopes.

Parameter	Range	Scope	Parameter	Range	Scope
rovers	2–16	session	annealer restarts	2–6	planner
scenario seed	0–99 999	session	cooling rate	0.980–0.999	planner
cruise speed	0–1.0 m/s	client	tasks per cycle	1–4	planner
time scale	0.25–3×	client	exploration	0–50%	planner
radio range	1.5–9 m	planner	trust memory α	0.10–0.95	planner
packet loss	0–60%	planner	reliability drift	0–0.10	planner
links per rover	1–6	planner	planning cycle	0.8–6 s	client
link budget	0–160 units	planner			

12.1.1 THE PLANNER SERVICE

Each browser mission owns a planner *session*. Alongside the primary simulator run three *paired followers*, one per assignment baseline (trust-only, random, no-graph), created with the same seed and copying the primary's kept links every cycle, so they differ from it *only* in the assignment policy; because they run from the start, a baseline chosen mid-mission is still paired from the same initial state. For a solver comparison, the exact Q matrix that the primary's annealer just solved, with the same warm start, is handed to the chosen sampler. Faults are applied to every arm. The service checks each field against the configuration contract. When it is already working it replies "busy" instead of putting the request in a queue, and when all session slots are full it drops the session that has been idle longest. On its side, the browser never has more than one request waiting. A slow solve therefore makes the cycles longer instead of letting requests pile up, and a reply meant for an old fleet can never be applied to a new one.

We picked the presets in Table 12.2 by measuring them. While calibrating, we found that with only one annealer restart some solvable instances ended up in the keep-all fallback (this is the restart-floor defect described in Section 5.7). Every preset therefore uses at least two restarts.

Table 12.2: The presets and the planner time per cycle we measured for each.

Preset	Rovers	Character	Cycle
Field survey	8	balanced radio budget, the everyday setting	680 ms
Rapid response	6	fast rovers, one-second planning cycle	280 ms
Relay corridor	10	long radios; the QUBO trims a dense mesh	750 ms
Dense swarm	12	short radios, faster cooling	780 ms
Full fleet	16	the maximum fleet size	1420 ms
Hostile radio	8	heavy packet loss, drifting reliability	250 ms
Tight constraints	10	two links per rover, small budget	140 ms

12.2 THE MOTION ENGINE

Fixed step and interpolation. The motion is simulated in fixed steps of $\Delta t = 1/120$ s, whatever the display's frame rate is. Each frame adds its real elapsed time (capped at 0.1 s, with a backlog cap of forty steps, so a stalled tab cannot trigger a burst of catch-up work) to an accumulator, advances by whole steps, and draws an interpolation between the last two steps with weight $\alpha = \text{accumulator}/\Delta t$. Headings are interpolated along the shorter arc, $\psi_0 + \alpha\,\text{wrap}(\psi_1 - \psi_0)$, so a heading that crosses $\pm\pi$ is never drawn spinning the long way round.

Exact arc integration. The ground plane is spanned by x and z, and a rover with heading ψ faces the direction $(\cos\psi, \sin\psi)$, so a positive yaw rate is a right turn. For constant v and ω over one step the rover follows a circular arc exactly, whose chord has length $v\Delta t\,\text{sinc}(\omega\Delta t/2)$ along the mean heading:

$$\begin{aligned} \begin{pmatrix} x' \\ z' \end{pmatrix} &= \begin{pmatrix} x \\ z \end{pmatrix} + v\Delta t\,\text{sinc}\!\left(\tfrac{\omega\Delta t}{2}\right) \begin{pmatrix} \cos(\psi + \omega\Delta t/2) \\ \sin(\psi + \omega\Delta t/2) \end{pmatrix}, \\ \psi' &= \text{wrap}(\psi + \omega\Delta t), \end{aligned} \tag{12.1}$$

with sinc evaluated by its Taylor series near zero. Unlike an Euler step, this update does not drift on curves.

Wheels and limits. Because a positive yaw rate turns right, the left wheel runs on the outside of the arc: $v_L = v + \omega L_w/2$ and $v_R = v - \omega L_w/2$, Eq. (11.1) rewritten for the mirrored yaw convention. An early prototype kept the Gazebo signs in this frame and spun the wheels backwards on every turn; a test that integrates each wheel's contact point and compares its path with $v_{L,R}\Delta t$ now pins the relation. Commands respect acceleration (0.9 m/s^2), braking (1.8 m/s^2), yaw rate (1.8 rad/s) and rim speed (1.25 m/s). An overfast command is scaled as $(v, \omega) \leftarrow \kappa(v, \omega)$ with $\kappa = \min(1, v_{\max}/\max(|v_L|, |v_R|))$, which preserves the curvature ω/v: the rover follows the same arc, only slower.

Steering. Each rover drives towards its task site or, when idle, a patrol point. The desired heading adds to the goal direction a repulsion from rovers within a sensing radius, weighted by $((S - d)/(S - 2R_b))^2$; a *keep-right* term for rovers ahead, so two rovers meeting head-on veer the same way and pass; an inward push at the arena edge; and, for idle rovers, a pull towards the nearest neighbour when it drifts beyond 85% of the radio range, which keeps the mesh connected. The speed is multiplied by the square of the cosine of the heading error. When the error is close to $\pm\pi$, the rover keeps turning in the direction it was already turning, so that it does not swing back and forth.

12.2.1 COLLISION SAFETY

We enclose each rover's footprint in a circle of radius $R_b = 0.62$ m. Two rovers then never collide as long as their centres stay at least $2R_b$ apart.

Proposition 12.1 (Sequential commits never overlap)**.** *The rovers are moved one after another. The translation proposed for rover i is only accepted if, for each other rover j at its* current *position, the new distance meets $d'_{ij} \geq 2R_b$ or $d'_{ij} \geq d_{ij}$. If not, rover i just turns on the spot. Then after every step $d^{new}_{ij} \geq \min(2R_b, d^{old}_{ij})$ for every pair.*

Proof. For a pair $i < j$, rover i is checked against j's old position and rover j, moving later, against i's new position, so

$$d(p'_i, p_j) \geq \min(2R_b, d(p_i, p_j)), \qquad d(p'_i, p'_j) \geq \min(2R_b, d(p'_i, p_j)).$$

Putting the two inequalities together proves the claim. If a move is rejected, its inequality holds trivially. A rotation in place does not move the centre of a circle, so it is always safe; the same argument keeps every rover inside the arena. □

The "or no closer" clause lets rovers that start too close separate instead of locking. Safety alone can still freeze a crowd, so every two seconds a rover that has gained less than 0.15 m towards its goal, or has been blocked for more than 1.2 s, takes a *detour* through a temporary waypoint about 63° off its goal direction, alternating sides on repeated attempts; after five failed detours it abandons the errand. Rovers spawn by deterministic dart throwing with spacing $2R_b + 0.35$ m, falling back to a hexagonal lattice that is guaranteed to hold the largest fleet. Tests drive sixteen rovers at triple speed for 1 800 frames without a single overlap and check that a dense swarm keeps moving.

12.3 THE RENDERER

Figure 12.2 shows the default mission from the orbit and top cameras. A naive scene creates one mesh per rover part and one line per link, so its draw calls, each carrying

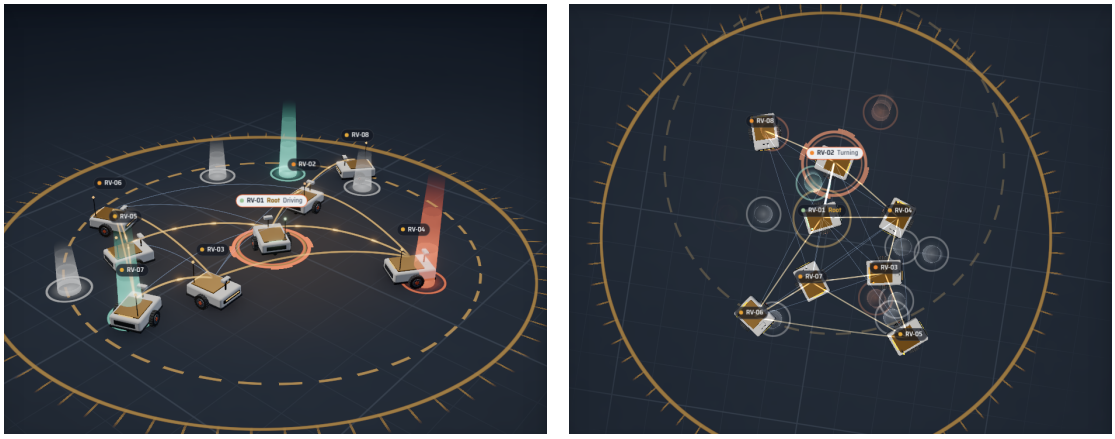


Figure 12.2: The eight-rover *Field survey* preset from the orbit camera (left) and the top camera (right): gold kept links, faint candidates, task beacons resolving to success (mint) or failure (orange), and the root's ring.

fixed CPU-side driver overhead, grow with the fleet. The engine instead renders every object class in one instanced draw call: the whole scene costs **seven draw calls** whatever the fleet size: the ground (one shader disc with grid, rim and fog), the ground markers (shadows, selection ring, radio ranges and beacon pads as instanced quads), the rover bodies (all parts merged into one geometry), the wheels (one geometry for both sides of every rover), the status lights, the radio links (instanced arc ribbons) and the task beacons. Eight rovers come to about 26 000 triangles. Per-instance data lives in pre-allocated buffers written in place and uploaded only over the range in use, so a frame allocates no memory.



Figure 12.3: Left: chase camera behind the default rover (ceramic body, gold-foil deck, stereo camera, antenna light, treaded wheels with orange hubs). Right: a four-wheeled model imported from Blender, whose named wheels were detected and spin with the fleet's kinematics.

The rover. The default rover reproduces the Gazebo model's dimensions and adds detail that makes heading and motion readable (Fig. 12.3). Parts of different colour and finish are merged into one geometry by storing colour, roughness and metalness per vertex and patching the standard physically based material to read them. Any rigid model exported from Blender as binary glTF can replace it: the import validates the file, centres

and grounds it, rotates its forward axis onto the engine's, scales its footprint to the collision circle, merges meshes per material, and detects meshes named as wheels, which then spin at $v_{L,R}/r$ for their own measured radius.

Radio links. Each link is a camera-facing ribbon bent into a shallow arc between two antennas, computed in a vertex shader so the CPU only supplies end points. For a link from A to B of length ℓ,

$$P(t) = A + t\,(B - A) + \hat{y}\,h\,4t(1 - t), \qquad h = \mathrm{clamp}(0.08 + 0.06\,\ell,\ 0.1,\ 0.5), \quad (12.2)$$

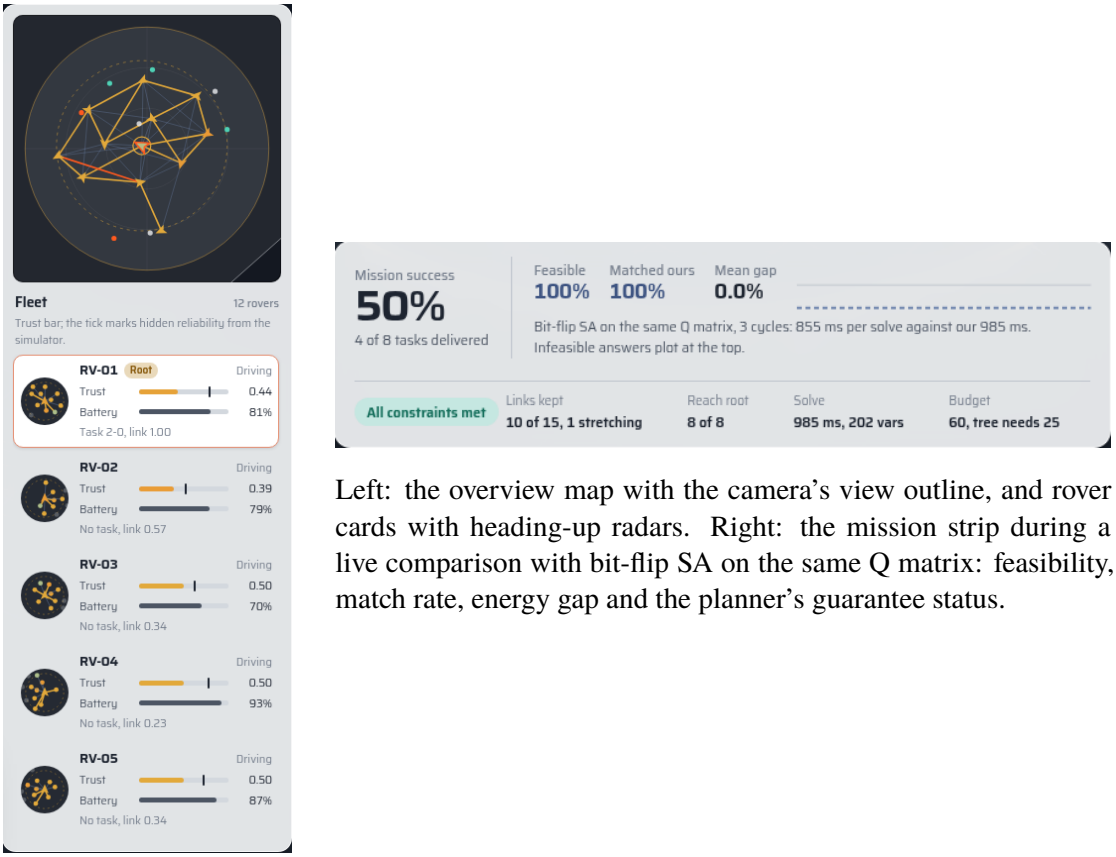
widened along the normalised cross product of $P'(t)$ and the direction to the camera, with a horizontal fallback when the camera looks along the link. Links are styled *kept* (gold), *candidate* (faint), *stretching* (flickering orange: kept, but out of range since the last plan) or *delivery path* (bright), and packets travel along kept links towards the root in the widest-path tree.

Ground and post-processing. The ground is one shader with an anti-aliased grid, the arena rim and the same linear fog as the standard materials. A post-processing chain adds anti-aliasing, bloom, filmic tone mapping and a vignette in three quality tiers, and an adaptive monitor steps down a tier when the frame rate falls. On one real GPU a screen-space ambient-occlusion pass produced black frames, because it could not read the multi-sampled depth buffer on that driver; the pass was removed, and a guard now samples five pixels after start-up and steps down a tier if all of them are black, which the ground and sky never are. A paused mission renders only on demand.

12.4 THE INTERFACE

Four cameras are available, orbit, top-down, follow and chase, with damped transitions that yield as soon as the user touches the controls. The right column (Fig. 12.4) is a game-style overview: a north-up map of the whole arena with links, task sites, every rover as an arrow coloured by trust, and the 3D camera's ground footprint, computed by intersecting the rays through the screen corners with the ground; below it, each rover's card shows a heading-up radar scaled to its radio range, its trust with a tick at its hidden true reliability, its battery and its task. The left panel holds presets, the comparison selector with the published reference result, fault injection, the tuning sliders with graph-theory advisories, overlays, quality tiers with live frame rate and draw calls, and model import.

The comparison selector lists only baselines that the published results show the pipeline beats (trust-only and random assignment, the no-graph readout marked as not significant, and the four samplers), and directly beneath it names the two methods that



Left: the overview map with the camera's view outline, and rover cards with heading-up radars. Right: the mission strip during a live comparison with bit-flip SA on the same Q matrix: feasibility, match rate, energy gap and the planner's guarantee status.

Figure 12.4: Overview map, rover radars and the live comparison strip.

beat the pipeline, the exact MILP and the threshold pruner. For assignment baselines the strip shows paired success rates, their difference and a bootstrap confidence interval, with a warning while fewer than fifteen cycles have been collected; for solver baselines it shows feasibility, match rate, mean energy gap and time per solve. In one 22-cycle run, bit-flip SA matched our optimum in 68% of cycles with a mean gap of 1.7%, in line with the offline benchmark. Short runs can go either way, and the engine shows them as they are.

12.5 EDGE CASES AND VERIFICATION

Table 12.3 lists the main edge cases and their treatment.

The motion mathematics is covered by seventeen property tests and the planner service by nineteen tests (Chapter 9). The engine was exercised in a browser at desktop, laptop and phone widths, where the panels reflow into one scrolling column.

Table 12.3: Edge cases handled by the engine.

Situation	Treatment
Degree cap 1, three or more rovers	a spanning tree needs a node of degree 2, so connectivity is impossible; announced before the planner relaxes it
Degree cap 2	only a chain through every rover (a Hamiltonian path) remains; relaxations announced
Budget below $n - 1$ units	every link costs at least one unit, so it can never be met
Budget versus the live graph	shown next to the cheapest spanning tree of the live graph
No links in range	reported as such, never as an empty success
Angles near $\pm\pi$; zero yaw rate	short-arc interpolation, held turn direction; Taylor series for sinc
Stalled tab; bad frame times	0.1 s frame cap, 40-step backlog cap, invalid deltas ignored
Malformed or stale planner data	invalid indices and values ignored; one request in flight; old-fleet answers aborted
Planner busy, full or offline	retry, idle-session eviction, or exponential back-off with a notice
GPU context lost; black post-processing	explanatory overlay; five-pixel guard steps down a tier

Chapter 13

Conclusion and Future Work

13.1 CONCLUSION

This project set out to allocate tasks in a rover fleet whose communication graph changes every few seconds, using a constrained QUBO to prune the graph and a trust-aware GNN to assign tasks on what remains. It delivered that pipeline, and it delivered something more valuable than the pipeline alone: a careful account of what each part contributes, measured against the strongest alternatives available.

For the optimisation part, we derived the QUBO encoding and checked it against an independent re-derivation and against exhaustive enumeration. A five-rover example showed that, because of how it is built, the encoding rejects the connectivity traps that a pruner using only soft scores would fall into. The solver audit then showed that a correct encoding is not enough on its own. Four well-known annealing-type samplers found the exact optimum on only 48 to 64% of the instances we could verify. The penalty walls that make a violation unprofitable also make it very hard for these samplers to cross from one feasible state to another. Once the search was restricted to feasible states, this barrier disappeared. The projected annealer is exact on all of the verifiable instances and stays within 1.4×10^{-3} of the exact optimum on all of the larger ones. Last, the exact MILP baseline is exact as well and about 100 times faster. For this reason we see the QUBO as an encoding for quantum hardware and not as the best way to solve the problem on a classical computer.

For the learning part, trust-aware assignment raises mission success by 8.1 points compared with a trust-only baseline, and the confidence interval for this gain does not include zero. Against random assignment the gain is 13.8 points. These results come from a simulator in which success depends causally on both the assignment and the pruned topology. Message passing adds a small margin that is not yet significant, and a cloud-trained readout did not beat the fixed weights on the data available.

On the systems side, the pipeline runs unchanged in a containerised Gazebo Harmonic and ROS 2 Jazzy world against live odometry, and in an interactive 3D engine whose rovers drive with exact, provably collision-free differential-drive kinematics, whose renderer costs a constant seven draw calls, and whose comparisons against weaker baselines are paired and statistically annotated.

13.2 KEY TAKE-AWAYS

1. **Bit-level annealers may fail even on a correct QUBO.** Before drawing any conclusion about a formulation, the solver's answers should be compared with an exact solution.
2. **On classical hardware, solve the constrained problem directly.** The MILP gives the exact answer and is two orders of magnitude faster. The QUBO is only worth using as an input format for annealing hardware.
3. **The guarantees are not free.** The constrained pruner is the only arm that never breaks a constraint, but at ten rovers it loses 2 to 4 points of raw success, and none of the pressure levels in our sweep turned the guarantee into higher success. It makes sense to use it when breaking a constraint is not acceptable at all.
4. **Most of the assignment gain comes from trust.** The trust model, the gates and delivery awareness give most of the improvement, and message passing adds only a small amount on top.
5. **Audit the audit.** Logging which constraints were actually enforced exposed a silently relaxed budget in the project's own earlier results, and changed its headline table.

13.3 PERFORMANCE AGAINST THE OBJECTIVES

Table 13.1 compares the outcomes with the objectives set in Chapter 1.

Table 13.1: Outcome of each objective.

Objective	Outcome
1. Constrained QUBO	Achieved: utility, stability, degree, budget and exact flow-based connectivity in one energy
2. Verified encoding	Achieved: agreement within 2.3×10^{-10}; no violated exact optimum in 38 instances
3. Reliable solver	Achieved with a caveat: projected annealer exact on all verifiable instances; generic samplers are not
4. Exact baselines	Achieved: direct MILP and exact linearised solve; MILP is faster than any annealer
5. Trust-aware GNN	Achieved: +8.1 pts over trust-only; message-passing margin and trained readout not significant
6. Causal evaluation	Achieved: causal delivery model, repaired baselines, pressure sweep, paired statistics
7. Deployments	Achieved: Gazebo and ROS 2 integration; interactive 3D engine with live comparisons

It is also worth being clear about what we built ourselves and what we used from others. We built the QUBO formulation and its verification, the projected annealer, the causal simulator, the evaluation protocol, the planner node, the planner service and the 3D engine. The samplers, the CBC solver, Gazebo, ROS 2 and the rendering libraries were used as published.

13.4 BENEFITS

Apart from the pipeline itself, the work is useful in three ways. The first is the way it evaluates things: exact referees, repaired baselines, causal metrics, paired statistics and a record of which constraints were relaxed can be reused for any QUBO-based graph method. The second is a hardware experiment that is ready to run, because the exported Q matrices and their certified optima form a benchmark on which quantum annealers can be compared fairly. The third is a tool for teaching and demonstrations, since the 3D engine lets people see constrained optimisation, trust learning and the trade-offs between them without having to read through tables of numbers.

13.5 FUTURE WORK

1. **Moving rovers in Gazebo.** Publish velocity commands from the planner node, using the controller of the 3D engine, so that the candidate graph changes in the physics simulation as it does in the browser.
2. **Physical validation.** Build the four-rover ESP32 testbed and replace the synthetic reliability and radio models with measured ones.
3. **Quantum annealing hardware.** The exported Q matrices could be run on a real quantum annealer and the results compared with the certified optima. This would show whether transverse-field dynamics handle the penalty barrier better than the classical samplers did.
4. **Delivery-aware utility.** The pruner's utility ranks links by their own quality and misses links that delivery depends on; adding path-criticality to the objective addresses the 0.67 recall directly.
5. **Scalable encodings.** To handle hundreds of rovers, the dense matrix and the flow variables would need to be replaced by a sparse or hierarchical encoding.
6. **Richer trust and on-policy training.** Contextual trust estimates and a larger exploration budget for training the readout.
7. **Statistical power.** More seeds and scenarios to resolve the small margins, such as message passing over the no-graph readout.

APPENDICES

Appendix 1: Pseudo-Code of the Pipeline, the Solver and the Motion Engine

This appendix gives, in algorithmic form, the per-cycle pipeline of Chapter 3 and the projected edge annealing of Chapter 5. The relaxation ladder and the motion engine's step are described in Sections 5.7 and 12.

Algorithm 2 One pipeline cycle

Input: telemetry for cycle t, trust T^{t-1}, previous kept edges

1: build G_t and the edge and rover features, using T^{t-1}
2: prune with projected annealing and the relaxation ladder (Section 5.7)
3: run two layers of trust-weighted message passing on the kept graph
4: score every rover for every task with gates, criticality and delivery factor
5: assign tasks greedily under per-rover capacity; explore with probability ε
6: observe execution and delivery outcomes
7: update trust by EWMA for assigned rovers and by aging for the others

Output: kept edges, assignment and T^t

Algorithm 3 Projected edge annealing

Input: QUBO builder, warm edge set W, schedule (τ_0, τ_f, γ), steps k, restarts n_r

best $\leftarrow$ none
for restart = $1, \ldots, n_r$ **do**
    $S \leftarrow W$ on the first restart, otherwise a degree-cap-aware random spanning tree
    if S cannot be projected to feasibility **then**
        continue
    end if
    $\tau \leftarrow \tau_0$
    while $\tau > \tau_f$ **do**
        for k times **do**
            form S' by switching one optional edge on or off, or by exchanging a kept edge for an unkept one
            if S' projects to feasibility (closed-form slacks, spanning-tree flow) **then**
                accept with probability $\min\{1, \exp(-(H(S') - H(S))/\tau)\}$
            end if
        end for
        $\tau \leftarrow \gamma\, \tau$
    end while
    polish: alternate single-toggle and pairwise-swap passes to a fixed point
    keep S if it is the best so far
end for
if best is none **then**
    fail: no feasible edge set
end if

Output: best edge set and its feasible bit vector

Appendix 2: Constants and Settings

Table A2.1: Constants and default settings used in this report.

Quantity	Value
QUBO	
Utility weights (trust, link, battery, proximity, freshness, task)	0.25, 0.25, 0.10, 0.10, 0.15, 0.15
Sparsity penalty λ / stability penalty η	0.08 / 0.15
Degree and budget penalties ρ_d, ρ_C	100
Flow penalties ρ_F, ρ_{FC}	500
Cost quantisation	$c_e = \max(1, \lfloor 10\,\bar{c}_e \rceil)$
Solvers	
Schedule of the bit-level SA (default)	start $\tau = 10$, end $\tau = 10^{-3}$, cooling rate 0.98, 2 sweeps, 20 restarts
Projected annealer, default settings	start $\tau = 1.0$, end $\tau = 0.01$, cooling rate 0.998, 4 steps, 5 restarts, swap probability 0.5
Projected annealer, inside the closed loop	cooling rate 0.995, 2 steps, 2 restarts
GNN and trust	
Self / neighbour weight	0.8 / 0.2
Readout weights	0.60, 0.15, 0.10, 0.15
Gates (trust, battery, compute)	0.25, 0.15, 0.30
Critical-task scaling	$(0.5 + 0.5\,T_i)$ when criticality > 0.5
Delivery modulation	$(0.5 + 0.5\,p_i)$
EWMA retention α / aging δ / prior	0.5 / 0.1 / 0.5
Exploration ε	0.15
Probation freshness floor	0.4
Simulator	
Area, random-walk step	10×10, standard deviation 0.8
Packet loss, freshness constant	5%, 3 cycles
Threshold / top-K fraction	0.4 / 0.5
Bootstrap resamples / permutations	10 000 / 20 000
Gazebo world	
Physics step, real-time factor	1 ms, 1.0
Rover body, mass	$0.9 \times 0.65 \times 0.25$ m, 3 kg
Wheel radius, width, separation	0.16 m, 0.08 m, 0.76 m
Planner cycle, radio range	1.0 s, 5.0 m
Planner degree cap, budget	3, 25 units

Quantity	Value
3D engine	
Arena radius, footprint radius R_b	6.0 m, 0.62 m
Fixed step, frame cap, backlog cap	1/120 s, 0.1 s, 40 steps
Acceleration, braking	0.9 m/s^2, 1.8 m/s^2
Yaw rate, yaw acceleration, rim speed	1.8 rad/s, 5 rad/s^2, 1.25 m/s
Arrival radius, slow-down radius, heading gain	0.3 m, 1.1 m, 3.2
Detour trigger	progress $<$ 0.15 m in 2 s, or blocked $>$ 1.2 s
Telemetry rate, map rate, radar rate	5 Hz, 30 Hz, 12 Hz
Session capacity, idle eviction window	16 sessions, 15 s

REFERENCES

[1]. Ahanger, T. A., Dahan, F., Tariq, U., & Ullah, I. (2023). Quantum inspired task optimization for IoT edge fog computing environment. *Mathematics, 11*(1), 156.

[2]. Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network flows: Theory, algorithms, and applications*. Prentice Hall.

[3]. Ali, B. A., Gregory, M., & Li, S. (2023). Trust-aware task load balancing in multi-access edge computing based on blockchain and a zero trust security capability framework. *Transactions on Emerging Telecommunications Technologies, 34*(12), e4845.

[4]. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multi-armed bandit problem. *Machine Learning, 47*(2–3), 235–256.

[5]. Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012). Fog computing and its role in the Internet of Things. In *Proceedings of the MCC Workshop on Mobile Cloud Computing*.

[6]. D-Wave Systems. (n.d.). *Ocean SDK and dwave-neal* [Software documentation]. `https://docs.ocean.dwavesys.com/`

[7]. Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

[8]. Forrest, J., et al. (n.d.). *COIN-OR branch and cut (CBC)* [Computer software]. `https://github.com/coin-or/Cbc`

[9]. Glover, F. (1989). Tabu search, part I. *ORSA Journal on Computing, 1*(3), 190–206.

[10]. Glover, F., Kochenberger, G., & Du, Y. (2019). A tutorial on formulating and using QUBO models. *arXiv:1811.11538*.

[11]. Guo, J., Huang, G., Li, Q., Xiong, N. N., Zhang, S., & Wang, T. (2021). STMTO: A smart and trust multi-UAV task offloading system. *Information Sciences, 573*, 519–540.

[12]. Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[13]. Jij Inc. (n.d.). *OpenJij: Framework for the Ising model and QUBO* [Computer software]. `https://github.com/OpenJij/OpenJij`

[14]. Kadowaki, T., & Nishimori, H. (1998). Quantum annealing in the transverse Ising model. *Physical Review E, 58*(5), 5355–5363.

[15]. Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.

[16]. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680.

[17]. Koenig, N., & Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (Vol. 3, pp. 2149–2154).

[18]. Korsah, G. A., Stentz, A., & Dias, M. B. (2013). A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research, 32*(12), 1495–1512.

[19]. Li, J., Tian, S., Wu, R., Zhu, L., Zhao, W., Meng, C., Chen, L., Zheng, Z., & Yin, H. (2023). Less can be more: Unsupervised graph pruning for large-scale dynamic graphs. *arXiv:2305.10673*.

[20]. Liu, C., Ma, X., Zhan, Y., Ding, L., Tao, D., Du, B., Hu, W., & Mandic, D. P. (2024). Comprehensive graph gradual pruning for sparse training in graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems, 35*(10), 14903–14917.

[21]. Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics, 2*, 5.

[22]. Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics, 7*(66), eabm6074.

[23]. Peng, H., Gurevin, D., Huang, S., Geng, T., Jiang, W., Khan, O., & Ding, C. (2022). Towards sparsification of graph neural networks. In *Proceedings of the 2022 IEEE 40th International Conference on Computer Design (ICCD)* (pp. 272–279).

[24]. Sabater, J., & Sierra, C. (2005). Review on computational trust and reputation models. *Artificial Intelligence Review, 24*(1), 33–60.

[25]. Spielman, D. A., & Teng, S.-H. (2011). Spectral sparsification of graphs. *SIAM Journal on Computing, 40*(4), 981–1025.

[26]. three.js authors. (n.d.). *three.js: JavaScript 3D library* [Computer software]. `https://threejs.org`

[27]. Verma, N. K., & Naik, K. J. (2026). MAG-Q: Multi-agent actor-critic based GRL with quantum inspired forwarding for enhanced task offloading in clustered fog computing environment. *Expert Systems with Applications, 318*, 132044.

[28]. Wang, L., Huang, W., Zhang, M., Pan, S., Chang, X., & Su, S. W. (2022). Pruning graph neural networks by evaluating edge properties. *Knowledge-Based Systems, 256*, 109847.